
“Do Diffusion Models Dream of Electric Planes?”

Discrete and Continuous Simulation-Based Inference for Aircraft Design

Aurelien Ghiglinio^{1,2} Daniel Elenius¹ Anirban Roy¹ Ramneet Kaur¹ Manoj Acharya¹ Colin Samplawski¹
Brian Matejek¹ Susmit Jha¹ Juan J. Alonso² Adam D. Cobb¹

Abstract

In this paper, we generate conceptual engineering designs of electric vertical take-off and landing (eVTOL) aircraft. We follow the paradigm of simulation-based inference (SBI), whereby we look to learn a posterior distribution over the full eVTOL design space. To learn this distribution, we sample over discrete aircraft configurations (topologies) and their corresponding set of continuous parameters. Therefore, we introduce a hierarchical probabilistic model consisting of two diffusion models. The first model leverages recent work on Riemannian Diffusion Language Modeling (RDLM) and Unified World Models (UWMs) to enable us to sample topologies from a discrete and continuous space. For the second model we introduce a masked diffusion approach to sample the corresponding parameters conditioned on the topology. Our approach rediscovers known trends and governing physical laws in aircraft design, while significantly accelerating design generation.

1. Introduction

Conceptual aircraft design is a complex engineering design problem that involves numerous multi-physics interactions between components (Raymer, 2012). During this exploratory phase, domain experts must utilize the high degree of design freedom to meet performance goals and adhere to constraints. Therefore, one of the key challenges is the narrowing down of the degrees of freedom while meeting the objectives and requirements for the aircraft. To meet these desired objectives, designers must run many simulations while optimizing inputs and observing outputs. To make this approach more efficient, simulation-based inference

(SBI) addresses this by ‘inverting’ simulators, amortizing their cost, and using the resulting model to sample input parameters (Lueckmann et al., 2021). For electric vertical take-off and landing (eVTOL) design, there are multiple challenges to using SBI directly. Firstly, discrete changes in the topology alter the number of parameters required to define an aircraft. For example, a single-winged aircraft needs no parameters for a second wing. Secondly, our design problem consists of 144 discrete aircraft configurations, where each has a maximum of 136 parameters to infer. This real-world, high-dimensional posterior exceeds the complexity of existing SBI benchmark tasks (Lueckmann et al., 2021). Finally, we face the challenge of representing designs with varying numbers of components. We address this by encoding each design as a vectorizable tree structure.

In this paper, we successfully use SBI for the design of eVTOL aircraft. We introduce a new hierarchical sampling approach that cascades two diffusion models to sample eVTOL designs conditioned on their desired performance metrics. These performance metrics include values such as mass, lift, and drag. The first diffusion model, the Mixed Diffusion Transformer (MixeDiT), enables joint sampling of discrete design topologies and continuous observations. This model enables conditioning on the observations and sampling the topologies. The second stage diffusion model, which we call a Masked Diffusion Transformer (MaskeDiT), is conditioned on an aircraft topology and samples the joint of the observations and the parameters. This enables sampling the input parameters conditioned on a topology and an observation. Overall the MixeDiT-MaskeDiT architecture enables sampling of a full eVTOL design.

To evaluate our eVTOL designs, we use SUAVE (Lukaczyk et al., 2015), a modular open-source tool for conceptual aircraft analysis. As part of our approach, we generate data via a probabilistic program that samples tree-structured representations of the design. We also implement our own CAD-based interference checks and structural checks. Using SUAVE, we evaluate our MixeDiT-MaskeDiT model with quantitative analyses leveraging distributional measures and posterior predictive checks. Importantly, we also include qualitative analyses, where we look at multiple case

¹Computer Science Laboratory, SRI ²Aerospace Design Laboratory, Stanford University. Correspondence to: Aurelien Ghiglinio <aghig@stanford.edu>, Adam Cobb <adam.cobb@sri.com>.

studies and explore whether known hypotheses are captured by our modeling approach.

Contributions. This work introduces several advances to simulation-based inference (SBI), and demonstrates its use for a real aerospace design problem:

- Our MixeDiT-MaskeDiT architecture enables SBI over a design space of discrete and continuous parameters.
- **MixeDiT** introduces a diffusion-based architecture that jointly samples discrete and continuous parameters.
- **MaskeDiT**’s topology-conditioned masking supports inference over **variable-dimensional design spaces**, extending SBI **beyond fixed-structure simulators**.
- Our approach is the first to address the full **conceptual aerospace design pipeline**, a realistic, large-scale, tightly coupled multidisciplinary setting whose design space **exceeds the dimensionality** considered in prior SBI research (144 aircraft topologies, each with up to 136 continuous parameters).
- We include a comprehensive dataset of **30,276 eVTOL aircraft designs** that span multiple configurations. This complements the existing open-source simulation engine, SUAVE (Lukaczyk et al., 2015).¹

2. Related Work

Machine Learning for Design. Previous works in machine learning for engineering design have often focused on building benchmark datasets (Cobb et al., 2023; Elrefaie et al., 2024; Hong et al., 2025; Sung et al., 2025; Regenwetter et al., 2025; Karri et al., 2025), or modeling specific components of a larger cyber-physical system, such as the CFD analysis of a wing (Bonnet et al., 2022; Shen & Alonso, 2025) or propeller design (Vardhan et al., 2024; Smart & Alonso, 2023). In this paper, we address additional challenges that arise from using multi-physics simulators to design eVTOL aircraft. Unlike prior work, our approach makes topological changes to the aircraft design that affect the total number of design parameters.

Simulation-Based Inference. Conceptual design is inherently different from direct optimization. During the early stages of design, we search for multiple options that meet the desired criteria and not just a single ‘optimal’ design. As such, SBI is an ideal choice for design problems as it performs Bayesian inference to learn distributions of designs rather than point estimates (Cranmer et al., 2015). Recent success in SBI has come from neural-based approaches, where each approach looks to compensate for the missing analytical likelihood, by either directly estimating the likelihood (Papamakarios et al., 2019b), directly modeling the posterior (Papamakarios & Murray, 2016), or model-

ing the ratio two likelihood functions (Cranmer et al., 2015; Thomas et al., 2022; Hermans et al., 2020; Cobb et al., 2024). For design, we require conditioning on multiple subsets of parameters, and not just the traditional choice of the observations. For example, we may condition on a certain wing span (input) and a coefficient of drag (output). As such, we look to the Simformer architecture of Gloeckler et al. (2024), a diffusion-based SBI approach that learns the full joint distribution and allows conditioning on any subset of parameters at inference time. We therefore focus on the use of diffusion-based SBI with two additional challenges. First, unlike the Simformer’s use-cases, our eVTOL designs can vary in their number of parameters and therefore we must introduce a new masking scheme. Second, during the first stage of our design hierarchy, we need to sample topologies that are encoded as discrete parameters.

We thus extend on current SBI approaches in order to perform mixed continuous and discrete sampling. Scaling SBI to such high-dimensional simulators remains difficult, with typical benchmark problems near $D = 10$ (Lueckmann et al., 2017) and only a few larger cases (Dax et al., 2025; Deistler et al., 2025). Here, we push diffusion-based SBI to $D = 136$.

Discrete Sampling and Discrete Diffusion. While some prior works have explored SBI with discrete parameters (Greenberg et al., 2019), and discrete outputs (Boelts et al., 2022), none have done so to the extent that we explore in this paper. In order to be able to sample from discrete distributions, we look to discrete diffusion models. Usually focused on language modeling, the majority of prior works on discrete diffusion operate directly on discrete states by introducing a forward process that leverages a transition matrix (Austin et al., 2021; Lou et al., 2024; Shi et al., 2024; Sahoo et al., 2024). The task of the reverse process is to then sequentially unmask tokens to sample from the data distribution. However, when sampling discrete and continuous parameters for design there would seem to be an advantage to jointly sampling in the same continuous space. As such, a recent work by Jo & Hwang (2026) proposed a continuous diffusion model for language modeling that can outperform discrete diffusion on language for moderate data sizes. Therefore, we combine this approach with the work of Zhu et al. (2025) that couples actions and observations. We use this latter approach to couple discrete and continuous parameters for design.

3. Data Generation: Conceptual Aircraft Design in SUAVE

Simulation using SUAVE. SUAVE is a flexible aerodynamics and flight dynamics simulation tool, which is often used for early-stage conceptual aircraft design (Lukaczyk et al., 2015). SUAVE ingests an aircraft design and a mission

¹The code is available at: <http://github.com/SRI-CSL/mixed-masked-diffusion>

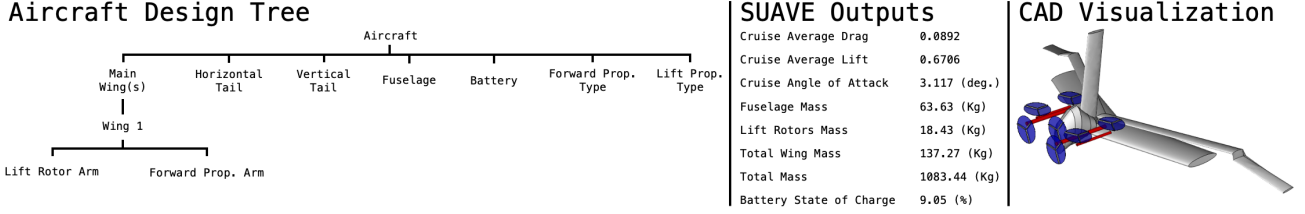


Figure 1. The left shows the tree of components for an aircraft with 1 main wing, two forward propellers and four lifting rotors. The middle shows the outputs from SUAVE, which is the design performance $\mathbf{x}$, for this aircraft. The right plot shows the CAD rendering of the same design as depicted in the tree. Refer to App. E, Fig. 19 for a more detailed image.

(flight path), and then performs three key coupled analyses sequentially: stability, aerodynamics and propulsion. After the analysis, SUAVE produces a large array of fine-grained aircraft statistics that can vary across the flight path, such as state-of-charge and angle of attack through different phases of flight. In this work, we will focus on a subset of ten summary statistics, including Cruise Average Drag C_D , Cruise Average Lift C_L , Cruise Angle of Attack α , and state of charge SoC . As a conceptual design tool, SUAVE relies on traditional mass-regression models (Raymer, 2012) and does not screen out structurally infeasible designs, so we build three modules. (1) A modified mass-estimation interface that supplies individual component masses to the simulation. (2) A CAD-based verification tool that detects propeller–wing or propeller–fuselage intersections. (3) A simple structural filter that rejects clearly infeasible configurations (e.g. wingspans over 50m). Additional details on SUAVE appear in App. E.

Design Representation and Generation. We generate a large design dataset by modeling eVTOLs as trees and using a custom tree-based probabilistic program. This probabilistic program plays the role of the general prior over all possible eVTOL designs. Starting from the root, each tree branches into propulsion, structural, aerodynamic, energy storage, and avionics subsystems. Each subsystem is further refined into concrete components, such as airframe members and wings, subcomponents, such as propeller rotors, until reaching atomic parameters, such as geometric dimensions and masses. This tree forms a typed, hierarchical JSON schema that functions like a generative grammar. An example of such a tree is shown in Fig. 1, with a full example shown in Fig. 19. Note that all designs include at least one wing, have one fuselage, and at least one forward propeller. We provide further details of our symbolic generative prior in App. E.

Design Stochasticity and Pre-Processing. The design space is constrained by capping the number of components: up to two wings, one horizontal stabilizer, one vertical stabilizer, two clusters of four lifting rotors per wing, two forward propellers per wing, and optionally a single nose-mounted propeller. Any combination of these elements is permitted,

resulting in 144 possible configurations. The final design representation for the machine-learning models is obtained by removing internal fuselage and wing cross-sections from the maximal design tree and flattening the remainder into a vector. Missing components (e.g., a second wing) are left blank in this vector, with a corresponding mask included in the dataset. We move the generation of internal fuselage and wing cross-sections to the run-time simulation, introducing stochasticity when decoding a pre-processed design. As a result, each design will have different internal fuselage and wing geometries for every SUAVE evaluation, while the key high-level parameters provided to the SBI model, such as total wingspan and fuselage length, remain fixed.

Dataset. Following the outlined data generation pipeline of this section, we initially generate a dataset of 300,000 eVTOL aircraft using our probabilistic program. We then run all designs through our SUAVE evaluation pipeline and filter out designs according structural checks, aircraft stability, and convergence of the SUAVE simulation. This results in a final dataset of 30,276 valid designs.

4. Hierarchical Mixed Diffusion

Notation. We define $\mathcal{T}$ to be the set of all possible UAV topologies, where $\tau \in \mathcal{T}$ defines all the discrete design choices for a particular aircraft. Example choices include number of wings, and the number of lifting rotors per wing. Our data includes 144 distinct eVTOL topologies. All observations are denoted as $\mathbf{x}$, where $\mathbf{x} \in \mathbb{R}^{D_{\mathbf{x}}}$ and $D_{\mathbf{x}} = 10$. Example observations include reported lift and drag coefficient. All continuous input parameters are denoted as $\boldsymbol{\theta} \in \mathbb{R}^{D_{\boldsymbol{\theta}}}$. Note that the dimension of $\boldsymbol{\theta}$ is dependent on the topology. The maximum size of $D_{\boldsymbol{\theta}}$ across all topologies is 126. For example, a two-winged eVTOL will require two sets of wing geometry parameters compared to a single-winged design. An overall design is given by $\Theta = \{\boldsymbol{\theta}, \tau\}$.

4.1. Overall Hierarchy

A key question in our design challenge is how to sample both a design topology and its corresponding set of parameters. Specifically, for a given topology τ_i , we have a set

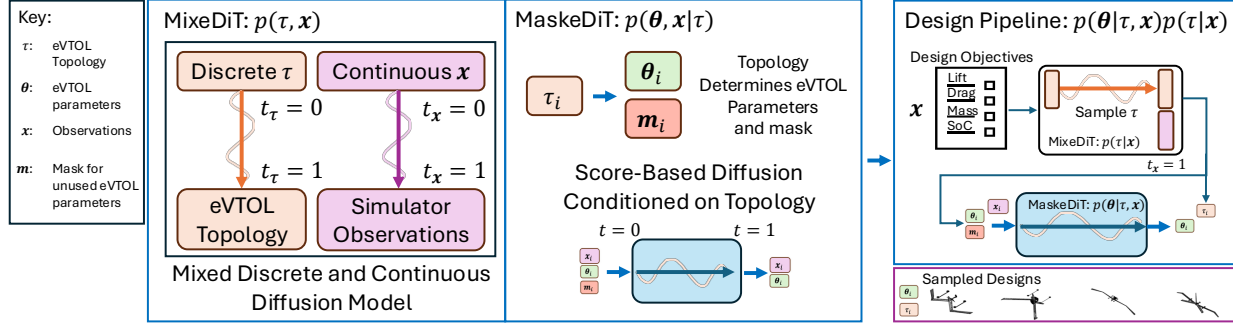


Figure 2. MixeDiT-MaskeDiT architecture. Left: overview of MixeDiT, which couples a discrete diffusion model over τ with a continuous diffusion model over $\mathbf{x}$. Middle: MaskeDiT, which uses topology masks $\mathbf{m}_i$ and design $\mathbf{x}$ to infer consistent design parameters. Right: full pipeline, where MixeDiT samples τ from $\mathbf{x}$, and MaskeDiT predicts θ from $(\tau, \mathbf{x})$.

of parameters θ_i of dimension D_{τ_i} which is not necessarily the same size as for a different topology τ_j . As such, a natural hierarchy emerges whereby one samples the topology first, followed by the corresponding parameters. Since our interest lies in the posterior over designs, we can formally write our hierarchal model as

$$p(\Theta|\mathbf{x}) = p(\theta|\tau, \mathbf{x})p(\tau|\mathbf{x}). \quad (1)$$

An advantage of this hierarchy is that for a given topology, the dimension of the distribution, $p(\theta|\tau, \mathbf{x})$, is fixed. This conditioning will facilitate the use of diffusion models without the need to account for variations in the size of the denoising process. We assume $\mathbf{x}$ is a fixed dimension, independent of the topology, where $\mathbf{x}$ includes high-level design observations such as the coefficients of lift and drag. Fig. 2 provides a summary of our approach.

4.2. MixeDiT: Distribution over the UAV Topology

To generate samples from $p(\tau|\mathbf{x})$, we use two modeling paradigms, RDLM (Jo & Hwang, 2026) and UWMs (Zhu et al., 2025), which enable joint sampling of topologies and observations. Our model learns to predict the denoising score of the observations, $\epsilon_{\mathbf{x}_t}^\omega$, and the time indexed probability of the topology, $\mathbf{p}_{\tau_t}^\phi$. We therefore define our model as $(\mathbf{p}_{\tau_t}^\phi, \epsilon_{\mathbf{x}_t}^\omega) = s_\phi(\tau_{t_\tau}, \mathbf{x}_{t_x}, t_\tau, t_x)$, which we call a Mixed Diffusion Transformer (MixeDiT).

Discrete Sampling in the Continuous Space. Our first step towards MixeDiT is to introduce the component responsible for the discrete sampling of the topology. We use RDLM to enable sampling using a continuous diffusion model. The RDLM models discrete diffusion via the continuous flow on a hypersphere. A categorical distribution is defined by its continuous probability simplex. Then the statistical manifold of the distribution corresponds to $\Delta^{d-1} = \{(p_1, \dots, p_d) \in \mathbb{R}^d | \sum_i p_i = 1, p_i \geq 0\}$ and is equipped with a Fisher-Rao metric. From this statistical manifold, the RDLM uses an existing diffeomorphism to go

from Δ^{d-1} to the positive orphant of a $(d-1)$ -dimensional sphere, on which distances are measured using the geodesic distance. The RDLM model, then uses prior work on Riemannian diffusion processes (Jo & Hwang, 2024) to build a diffusion model on this manifold. We leverage this approach due to the reported improved performance in discrete diffusion modeling. We also highlight the unexplored possibility of jointly sampling discrete and continuous parameters using the same continuous diffusion machinery, which is not possible with prior discrete diffusion models that restrict themselves to discrete Markov chains (see Sec. 2). This observation is one of our key contributions to enabling MixeDiT. Therefore, referring back to our new model definition, we introduce the RDLM importance sampled cross-entropy loss for the discrete head of our model:

$$\mathcal{L}^{RD}(\phi) = \mathbb{E}_{p(\tau^1)p(\tau^t|\tau^1, \tau^0)q(t_\tau)} \left[-\frac{1}{q(t_\tau)} \log \langle \mathbf{p}_{\tau_t}^\phi, \tau^1 \rangle \right], \quad (2)$$

where $\langle \cdot, \cdot \rangle$ indicates inner product, and τ^1 is a sample from the data distribution, where $\tau^1 \sim p(\tau)$ and defines a one-hot encoding of the topology. We use the superscript to emphasize the end point of the diffusion. The distribution over the interpolant, $p(\tau^t|\tau^1, \tau^0)$, leverages the simulation-free approximation (App. B.1). Note that $\mathbf{p}_{\tau_t}^\phi$ is a function of τ^t . We use the stepped importance weight for the time $q(t)$, which focuses on up-weighting the middle region of the diffusion (see App. B.1).

Coupling discrete and continuous diffusion. To connect the discrete and continuous diffusion components, we take inspiration from Zhu et al. (2025), where they integrated two independent continuous diffusion processes. Instead, we integrate a discrete diffusion process for the eVTOL topology, and a continuous diffusion for the observations. We first introduce the noise-prediction form of the conditional denoising score matching loss that we use for the observations:

$$\mathcal{L}^{CSM}(\phi) = \mathbb{E}_{p(\mathbf{x}_k)p(t_x)p(\epsilon)} [||\epsilon_{\mathbf{x}_t}^\phi - \epsilon||_2^2]. \quad (3)$$

and then we introduce the MixeDiT Loss as a weighted combination of the discrete and continuous losses,

$$\mathcal{L}^{\text{MixeDiT}}(\phi) = \gamma \mathcal{L}^{\text{RD}}(\phi) + (1 - \gamma) \mathcal{L}^{\text{CSM}}(\phi), \quad (4)$$

where $\gamma \in [0, 1]$ is the loss weighting parameter. To evaluate the loss, $\mathcal{L}^{\text{MixeDiT}}(\phi)$, the expectation is taken with respect to the data distribution, $p(\tau, \Theta)$, the two independent time distributions, $p(t_x)$, and $q(t_\tau)$, the noise $p(\epsilon)$, and the discrete interpolant $p(\tau_t | e_k, t)$. The full training algorithm is given in Alg. 1, App. A.

Sampling eVTOL Topologies. We can use our model to sample from the marginal distribution of topologies $p(\tau)$ by setting the observation timestep $t_x = 0$, and performing the geodesic random walk using the RDLM head:

$$\tau_{t+\delta t} = \exp_{\tau_t} \left(\eta_\phi \delta t + \sigma_t \sqrt{\delta t} \mathbf{w} \right), \quad \mathbf{w} \sim \mathcal{N}(0, \mathbf{I}_d), \quad (5)$$

where η_ϕ is the parameterization of the drift term on the Riemannian manifold (see App. B.1, Eq. (8)) and is a function of $\mathbf{p}_{\tau_t}^\phi$, and τ_t . The function, $\exp_{\tau_t}$, is the exponential map to the statistical manifold. At the end of the sampling, where $t_\tau = 1$, we implement the argmax to get the corresponding 1-hot representation of the topology, τ^1 . For conditional sampling, $p(\tau | \mathbf{x})$, based on a specific observation, $\mathbf{x}^*$, we set $t_x = 1$ and run the same sampling scheme as in Eq. (5). (See Alg. 2.)

4.3. MaskeDiT: Topology-Conditioned Sampling

For each design topology τ_i we sample, we have an additional $\theta_i \in \mathbb{R}^{D_{\tau_i}}$. This conditionally varying dimension presents itself as a challenge to traditional SBI approaches in the literature. To enable SBI over a high dimensional space, we leverage the Simformer architecture (Gloeckler et al., 2024), which directly models the joint distribution $p(\theta, \mathbf{x})$ via a score-based diffusion model, where the score is learned using a transformer. Unlike the original Simformer, which has a fixed variable structure, we introduce a Masked Diffusion Transformer (MaskeDiT) architecture. Our objective is to construct a model capable of representing a broad class of aircraft topologies, including arbitrary numbers and arrangements of components, parameterized by $\theta \in \mathbb{R}^{D_\tau}$, where the dimension, D_τ , is allowed to vary.

Diffusion Model Architecture. In Simformer, both the likelihood $p(\mathbf{x} | \theta)$ and posterior $p(\theta | \mathbf{x})$ are obtained by conditioning. Each data point is encoded as a sequence of tokens, where each token $\in \mathbb{R}^h$, where h is the token dimension, corresponds to a parameter θ or output $\mathbf{x}$, and is formed by concatenating an identity embedding, the feature value, and a condition-state embedding. Positional encodings are applied to the identity and condition-state vectors. Conditioning is enforced through two mechanisms: each

token is augmented with the aforementioned condition-state embedding indicating whether it is observed, and the same mask is applied inside the attention layers to suppress any attention edges that would transmit information from unobserved to observed values (or vice-versa). We call this condition mask M_C , with the condition denoted as C . Randomly sampled condition masks train the model across a family of conditional distributions. Further details are in App. C.

Masking Components: MaskeDiT. While the Simformer operates on a fixed-size token sequence $\theta \in \mathbb{R}^{d_{\max} \times h}$ that allocates a slot for every possible component in the design space ($d_{\max} = 136$), this fixed-sized graph prevents us from being able to vary D_τ according to the topology. Therefore we introduce a topology masking scheme, where a topology is encoded by a binary mask $M_\tau \in \{0, 1\}^{d_{\max}}$. $M_{\tau,i} = 1$ indicates that component i is present and $M_{\tau,i} = 0$ indicates that it is absent. During both attention and feed-forward computation, the mask suppresses nonexistent components by zeroing out their corresponding token embeddings, ensuring that only indices with $M_{\tau,i} = 1$ participate in the model's computation and training.

Training MaskeDiT. The topology enters the model in two ways. First, we apply $M_\tau \otimes M_\tau$ as an attention mask to restrict attention to existing components, where $\otimes$ is the outer product. Second, in the diffusion loss, terms with $M_{\tau,i} = 0$ are omitted, yielding a normalized loss $\sum_i \ell_i M_{\tau,i} / |\{i : M_{\tau,i} \neq 0\}|$. Conditioning acts similarly, with masked components applying in attention and in the loss, alongside the aforementioned conditional tokenizing. The condition masks used in training are changed in every iteration as in Gloeckler et al. (2024), with a uniform probability of choosing a mask corresponding to the joint, likelihood, posterior, random Bernoulli with $p = 0.3$ or random Bernoulli with $p = 0.7$. We therefore define a combined mask $M = (1 - M_C) M_\tau$. For training the diffusion model, instead of using the score matching loss, as originally proposed in Gloeckler et al. (2024), we found that the noise matching loss was more stable. Therefore, the loss for the MaskeDiT architecture is

$$\mathcal{L}^{\text{MaskeDiT}}(\omega) = \frac{M(\tau, C) \|\epsilon^\omega(\theta_t^{\tau, C}, \mathbf{x}_t^C) - \epsilon\|_2^2}{\sum_i M_i(\tau, C)} \quad (6)$$

The noising function is a gaussian probability path that is variance preserving (as in DDPM (Ho et al., 2020)). We use the exponential moving average (EMA) on the model parameters during training to increase training stability and sample quality (Ho et al., 2020; Brotons et al., 2024). We use an EMA decay rate of 0.999. The algorithm is described in Alg. 3, App. A and further details are in App. C.

Sampling from $p(\theta|\tau, \mathbf{x})$ The sampling process involves integrating the learned reverse-time SDE using the Euler-Maruyama integration scheme. Sampling begins by drawing from a Gaussian distribution $\mathbf{x}_0 \sim \mathcal{N}(0, I)$. At each step, the learned noise is converted to a score estimate by

$$\mathbf{s}^\omega(t) \approx \frac{\epsilon^\omega(\theta_t^{\tau, C}, \mathbf{x}_t^C)}{\beta(t)} \quad (7)$$

where $\beta(t)$ is the standard deviation of the noising process. The full algorithm is in Alg. 4, App. A. As in Gloeckler et al. (2024), known values immediately overwrite the corresponding $\mathbf{x}$ values at each step of the denoising process to ensure exact conditioning. The condition C and topology τ are fixed throughout sampling, and are user-defined inputs. Note that though we do not use guidance for conditioning here, guidance could be used as shown in Gloeckler et al. (2024).

5. Results

We first evaluate each individual diffusion model separately, before evaluating their joint performance in Section 5.3.

5.1. MixeDiT: Sampling eVTOL Topologies

Marginal Sampling. We first quantify MixeDiT performance by comparing the test data’s marginal distribution $p_D(\tau)$, with marginal samples generated via the sampling procedure of Eq. (5) with $t_x = 0$. Fig. 3 plots these two empirical distributions on the log probability scale across the full vocab size of UAV topologies (144). MixeDiT achieves a KL-divergence of $D_{KL}(p_D(\tau)||p_\phi(\tau)) = 0.0505$, showing strong performance in capturing the marginal distribution. For comparison, $D_{KL}(p_D(\tau)||\mathcal{U}(\tau)) = 1.4607$, highlighting the complexity of the distribution.

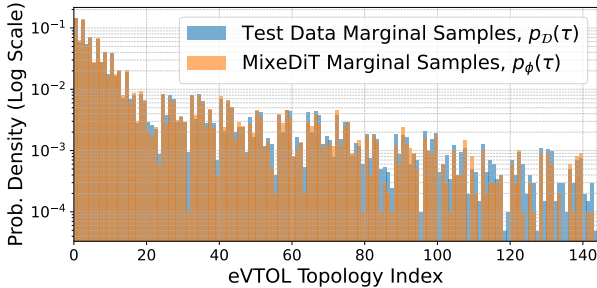


Figure 3. Comparison of the marginal test distribution of the topologies vs. the marginal samples generated by MixeDiT. The log scale highlights the performance on low-probability topologies.

Posterior Sampling. For conceptual design, we want to propose eVTOL topologies conditioned on design objectives. As such, we now implement the sampling procedure of Eq. (5) with $t_x = 1$, and $\mathbf{x}$ set to a desired value. Fig. 4 high-

lights two conditioning scenarios, as well as the data and MixeDiT marginal distributions. The graph highlights summary statistics of the eVTOL topologies, such as whether a second wing is present. For the conditioning scenarios, we compare a “**high mass**” example with a “**low mass**” example. For the **high mass** example, we set the mass observation to be two standard deviations greater than the data mean ($x_{mass} = \mu_{mass} + 2\sigma_{mass}$). For the **low mass** example we reduce the mass to two standard deviations below the mean ($x_{mass} = \mu_{mass} - 2\sigma_{mass}$). All other observations are set to the data mean ($\mathbf{x}_{/mass} = \mu_{D/mass}$). One of the key outcomes is that 91.6% of the samples that were conditioned on the higher mass contained a second wing (green bar). This compares to the 0.4% of two-winged designs for the lower mass samples (red bar). This result agrees with the expected behavior that increasing the mass of our designs should lead to additional wings, which add both weight and lift.

5.2. MaskeDiT: Likelihood over Parameters Conditioned on Topology

Table 1. Likelihood comparison between generated samples $p_\phi(\mathbf{x}|\theta)$ and test data $p_D(\mathbf{x}|\theta)$. Average is taken over 10 topologies with more than 200 samples in the test dataset.

METRIC	MASKEDI T	NLE
MMD VALUE	0.004 ± 0.001	0.002 ± 0.001
JOINT C2ST	0.777 ± 0.040	0.744 ± 0.023
MEAN MARG. C2ST	0.614 ± 0.016	0.596 ± 0.010
MAX MARG. C2ST	0.770 ± 0.076	0.818 ± 0.043

Metrics. We evaluate our approaches using two metrics. The first metric is the maximum mean discrepancy (MMD) (Gretton et al., 2012), which we aim to minimize. The second metric, classifier two-sample test (C2ST) (Lopez-Paz & Oquab, 2017), trains a binary classifier to distinguish real from generated samples, where a score of 0.5 indicates indistinguishable distributions. We report C2ST for both joint and marginal distributions (Tab. 1). We remove strongly correlated observation variables whose samples collapse onto a near-linear relationship, as C2ST is not informative in these cases. We use a Pearson correlation of greater than 0.9 to exclude these from the C2ST computation.

Analysis. Tab. 1 compares MaskeDiT with Neural Likelihood Estimation (NLE) (Papamakarios et al., 2019a), using the implementation from the sbi package (Tejero-Cantero et al., 2020). Because NLE trains a separate model for each topology, it can specialize and achieve lower MMD on individual topologies, but it cannot scale across the 144 topologies in our design space. On the other hand, MaskeDiT must generalize across all topologies with a single model, learning over a substantially larger mixed discrete-continuous space. Although NLE achieves a lower

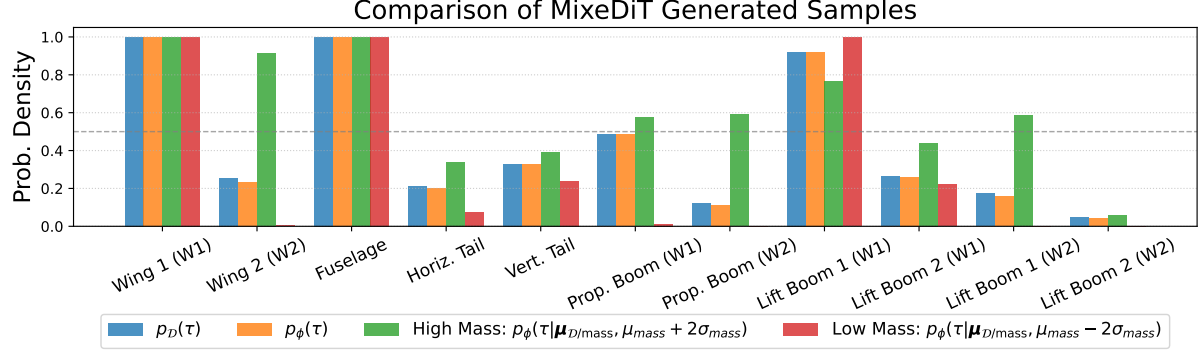


Figure 4. Summary statistics of generated samples using MixeDiT. The original data distribution (blue) closely matches the MixeDiT marginal generated samples (orange). Conditioning on higher mass designs (green, $+2\sigma_{mass}$) significantly increased the number of two-winged designs, while conditioning on a low mass design dramatically reduces the number of two-winged designs (red, $-2\sigma_{mass}$).

average MMD, MaskeDiT maintains a similarly low MMD while attaining comparable C2ST scores, demonstrating strong multi-topology generalization with only a modest tradeoff in per-topology fidelity. Max marginal C2ST further shows that MaskeDiT maintains more uniform performance across variables, achieving a lower worst-case marginal discrepancy than NLE. The full table of metrics is provided in App. H.

Fig. 5 provides a comparison of metrics across-topologies, including rare topologies. NLE degrades sharply when trained on very small datasets, producing NaN MMD values and C2ST scores of 1.0, whereas MaskeDiT remains stable even for rare topologies. For topologies with roughly 400 to 1500 training samples, the two methods achieve similar C2ST values, while MaskeDiT exhibits more uniform performance across variables (lower worst-case marginal C2ST). Overall, these results reinforce that MaskeDiT provides strong multi-topology generalization with only a modest tradeoff in per-topology fidelity, while enabling conditional flexibility and scalability that NLE cannot support.

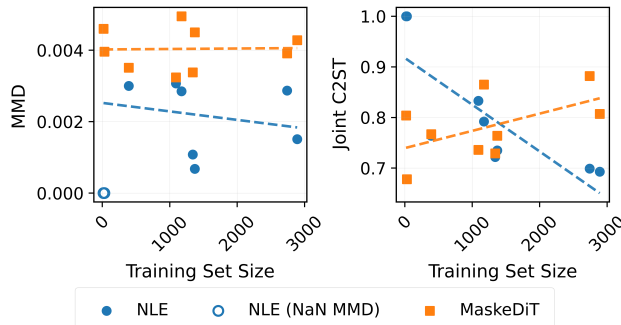


Figure 5. Per-topology MMD and joint C2ST scores for nine representative topologies, illustrating performance variation under the varying per-topology frequency of the training set. NLE becomes unstable at topologies with very small training sizes, whereas MaskeDiT maintains consistent performance across the full range.

5.3. Full Pipeline

5.3.1. POSTERIOR PREDICTIVE PERFORMANCE

In order to verify the posterior performance, we sample the topology using MixeDiT, conditioned on fixed $\mathbf{x}$ values. We then sample the corresponding continuous parameters using MaskeDiT. After generating these designs, we simulate them using SUAVE, where the distributions of observations from the simulator can be contrasted to the input $\mathbf{x}$ values. Conditioning on the mean $\mathbf{x} = \mu$, as well as $\mathbf{x} = \mu \pm \sigma$, Fig. 6 shows that in all cases the posterior distribution’s mean agrees closely to the input observed value. Further details are presented in App. F.

MixeDiT-MaskeDiT vs. SUAVE Simulation. The wall-clock time of the full MixeDiT-MaskeDiT pipeline for unconditional sampling is $\geq 10\times$ versus running the SUAVE simulation for 100 samples. The speedup is more significant for larger batch sizes. A description of the hardware is in App. G.

5.3.2. TESTING KNOWN DESIGN HYPOTHESES ACROSS TOPOLOGIES

We now include multiple case studies to explore the generated samples from the MixeDiT-MaskeDiT distributions. Additional analysis for each study is included in App. D.

Case Study A: Higher weight for a fixed topology, fixed state of charge and fixed drag.

Hypothesis. Increasing the weight requires the production of *more lift*, while constraining state of charge requires the aircraft to remain efficient. This could manifest in a *greater wingspan*, which would lead to decreased induced drag and an *increase in wing area*. We would also expect the *battery mass to increase* in order to power the larger aircraft.

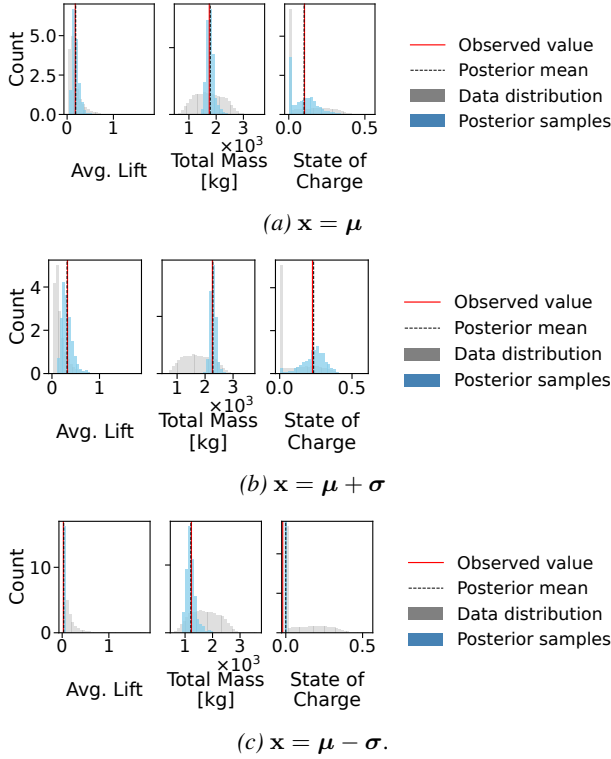


Figure 6. Posterior predictive distributions of selected x variables under three conditioning scenarios.

We see a few clear trends from Fig. 7. First is the increase in battery mass with aircraft mass in order to achieve the final state of charge required. We also see the expected increase in wing area and hence mass. In the training dataset, the lifting rotor design is typically limited by the speed of the blade tip. This leads to shorter rotor blades as the mass, and hence rotor RPM, increases.

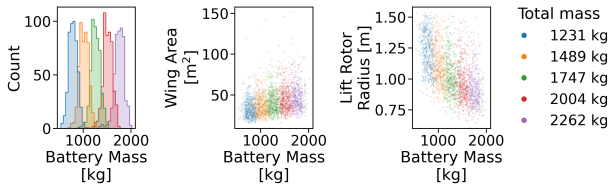


Figure 7. Case Study A: Trends in selected variables for increasing total eVTOL mass with fixed topology, drag and state of charge.

Case Study B: Increasing the lift of monoplanes versus biplanes for a fixed wing mass. In this case we take advantage of the masking diffusion model to directly compare different topologies.

Hypothesis. Drag should increase with increasing lift with both topologies, though the biplanes should on average have a greater total wingspan.

In Fig. 8, we notice several trends. Overall, we see clear clustering both by weight and by topology. Lower lift leads to lower battery mass designs, regardless of topology. The near-quadratic relationship between lift coefficient and drag coefficient is recovered - a well known physical approximation saying that drag is approximately proportional to the square of lift ($C_D = C_{D,0} + KC_L^2$). However, the total wingspan of the sampled biplane designs is greater than the total wingspan of the monoplane designs.

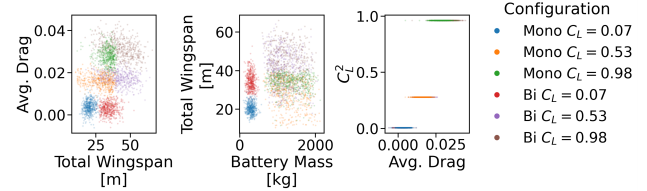


Figure 8. Case Study B: Trends in selected variables for biplane (Bi) and monoplane (Mono) eVTOLs for increasing lift (C_L) while keeping wing weight fixed.

When considering the 3D renders of samples in Fig. 9, we see that although the wings become skinnier, in order to increase the lift coefficient while maintaining wing mass, we see the wing span increase as in Case A.

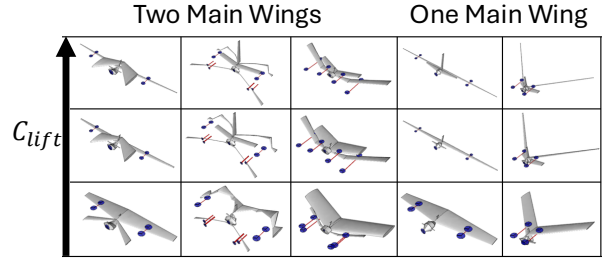


Figure 9. Case Study B: parameter variations when considering biplane and monoplane topologies.

Case Study C: Increasing total mass with all other x fixed. In this task, we are conditioning on the outputs, and seeing how this leads to both topological and parameter variation, which is a capability unique to our architecture.

Hypothesis. We would require greater battery mass and greater wing mass to lift a heavier aircraft.

Fig. 10 provides a snapshot of the results of Case Study C. We see the expected trend that battery mass increases with total mass. Furthermore, increase in total mass is correlated with an increase in the wing mass. We see the lift rotor blade lengthening, reflecting the need for greater rotor area to support heavier designs. The cross correlation between battery mass and lift rotor radius has two clear clusters, one for monoplanes and one for biplanes. The

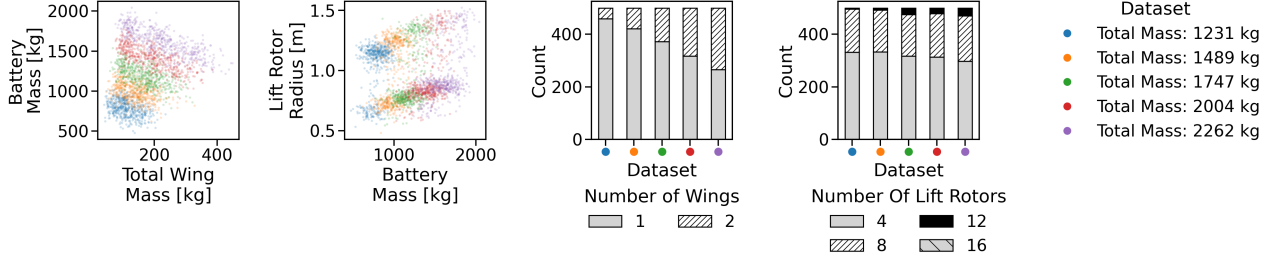


Figure 10. Case Study C: Trends in selected variables for increasing eVTOL total mass while keeping other observations such as lift, drag, wing mass and state of charge fixed.

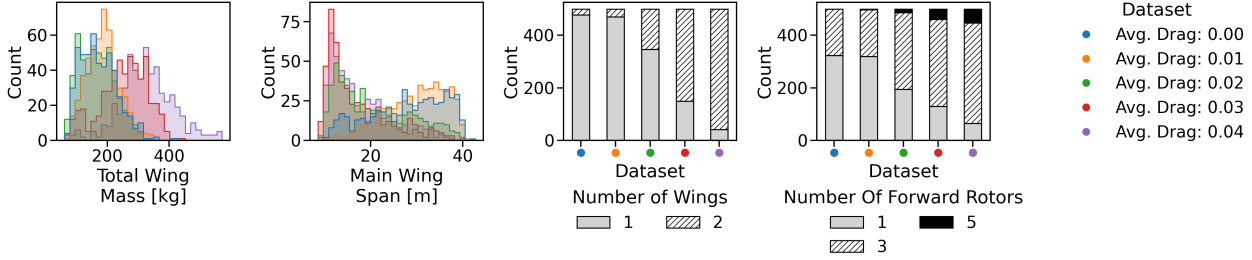


Figure 11. Case Study D: Trends in selected variables for increasing drag while keeping the remaining observations (e.g. total mass, lift, state of charge) fixed.

number of lift rotors increases with mass to maintain the vertical climb efficiency (figure of merit). Heavier designs more frequently exploit the additional lift provided by a second wing, although a single-wing configuration remains the predominant choice overall.

Case Study D: Increasing drag with all other x fixed.

Hypothesis. For greater drag with the same lift, we would expect *smaller wingspans* to increase induced drag. Designs with greater drag should have *more components*.

Fig. 11 provides a summary of the results for Case Study D. For a greater drag, we see an increase in the number of forward rotors, as this increases rotor-wing interaction effects and the profile drag. This is shown in the rightmost plot. In the plot second from the right, we also see that draggier designs typically have a second wing, due to the greater profile drag (surface friction), and, depending on the wing arrangement, greater induced drag (lift-related). In Fig. 11, we see the decreased wingspans predicted, as well as increased wing weight.

The renderings of aircraft samples in Fig. 12 show clear trends with increasing drag coefficient. Induced drag is increased as wings go from long and skinny to thicker and stubbier. The number of lifting rotors increases as these are dead weight during cruise. Designs with greater drag have a greater number of wings stacked vertically close together.

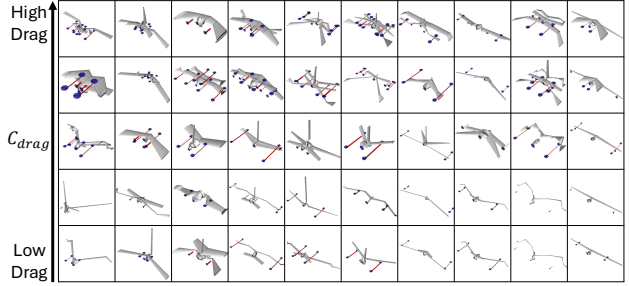


Figure 12. Case Study D: rendered topology changes while conditioning on a range of drag coefficients.

6. Conclusion

In this work, we introduced MixeDiT and MaskeDiT, which are two diffusion-based models, composed hierarchically, that enable SBI over a mixed continuous–discrete, and variable-dimensional design space. Our approach accurately captures the full distribution of topologies, including low-probability configurations, and achieves strong posterior agreement between performance-conditioned samples and simulator predictions. Using the MixeDiT-MaskeDiT architecture, we are able to recover key aerospace design trends: battery mass scales with aircraft mass, wing configurations adapt to increasing lift requirements, and rising drag induces predictable topology changes such as additional wings. These results demonstrate that MixeDiT and MaskeDiT deliver interpretable and scalable SBI performance in complex engineering design settings.

Impact Statement

AI-powered generative design offers opportunities to benefit broader society. By enabling lighter, more efficient aircraft, we can align with global responsibilities to minimize environmental impact and promote sustainable aviation while reducing costs. By expanding the design space, generative design may enable designers to discover new concepts and mitigate for previously overlooked risks. However, as with all automation, the introduction of less transparent algorithmic processes also raises concerns about accountability and unintended design biases. Future iterations of this work should be mindful of the potential for conceptual work to be used in detrimental downstream aerospace applications.

References

- Austin, J., Johnson, D. D., Ho, J., Tarlow, D., and Van Den Berg, R. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021.
- Boelts, J., Lueckmann, J.-M., Gao, R., and Macke, J. H. Flexible and efficient simulation-based inference for models of decision-making. *Elife*, 11:e77220, 2022.
- Bonnet, F., Mazari, J., Cinnella, P., and Gallinari, P. Airfrans: High fidelity computational fluid dynamics dataset for approximating reynolds-averaged navier–stokes solutions. *Advances in Neural Information Processing Systems*, 35: 23463–23478, 2022.
- Botero, E. M., Clarke, M. A., Erhard, R. M., Smart, J. T., Alonso, J. J., and Blaufox, A. Aerodynamic verification and validation of suave. In *AIAA SciTech 2022 Forum*, pp. 1929, 2022.
- Brotons, D. M., Vogels, T., and Hendrikx, H. Exponential moving average of weights in deep learning: Dynamics and benefits. *Transactions on Machine Learning Research Journal*, pp. 1–27, 2024.
- Clarke, M. and Alonso, J. J. Lithium–ion battery modeling for aerospace applications. *Journal of Aircraft*, 58(6): 1323–1335, 2021.
- Cobb, A., Roy, A., Elenius, D., Heim, F., Swenson, B., Whittington, S., Walker, J., Bapty, T., Hite, J., Ramani, K., et al. Aircraftverse: a large-scale multimodal dataset of aerial vehicle designs. *Advances in Neural Information Processing Systems*, 36:44524–44543, 2023.
- Cobb, A. D., Matejek, B., Elenius, D., Roy, A., and Jha, S. Direct amortized likelihood ratio estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 20362–20369, 2024.
- Cowden, D. and CadQuery Contributors. CadQuery: A python-based parametric cad scripting framework. URL <https://doi.org/10.5281/zenodo.3955118>.
- Cranmer, K., Pavez, J., and Louppe, G. Approximating likelihood ratios with calibrated discriminative classifiers. *arXiv preprint arXiv:1506.02169*, 2015.
- Dax, M., Green, S. R., Gair, J., Gupte, N., Pürner, M., Raymond, V., Wildberger, J., Macke, J. H., Buonanno, A., and Schölkopf, B. Real-time inference for binary neutron star mergers using machine learning. *Nature*, 639(8053): 49–53, 2025.
- Deistler, M., Boelts, J., Steinbach, P., Moss, G., Moreau, T., Gloeckler, M., Rodrigues, P. L., Linhart, J., Lappalainen, J. K., Miller, B. K., et al. Simulation-based inference: A practical guide. *arXiv preprint arXiv:2508.12939*, 2025.
- Elrefaie, M., Morar, F., Dai, A., and Ahmed, F. Drivaernet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks. *Advances in Neural Information Processing Systems*, 37:499–536, 2024.
- Gloeckler, M., Deistler, M., Weilbach, C. D., Wood, F., and Macke, J. H. All-in-one simulation-based inference. In *International Conference on Machine Learning*, pp. 15735–15766. PMLR, 2024.
- Gratz, J., Kirk, J., Recine, C., Jasa, J., Aretskin-Hariton, E., and Moore, K. Aviary: An open-source multidisciplinary design, analysis, and optimization tool for modeling aircraft with analytic gradients. URL <https://github.com/OpenMDAO/Aviary>. NASA Glenn Research Center; NASA Langley Research Center; NASA Ames Research Center; Banner Quality Management Inc.
- Greenberg, D., Nonnenmacher, M., and Macke, J. Automatic posterior transformation for likelihood-free inference. In *International conference on machine learning*, pp. 2404–2414. PMLR, 2019.
- Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., and Smola, A. A kernel two-sample test. *The journal of machine learning research*, 13(1):723–773, 2012.
- Hermans, J., Begy, V., and Louppe, G. Likelihood-free MCMC with amortized approximate ratio estimators. In *International conference on machine learning*, pp. 4239–4248. PMLR, 2020.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

- Hong, S., Kwon, Y., Shin, D., Park, J., and Kang, N. Deep-jeb: 3d deep learning-based synthetic jet engine bracket dataset. *Journal of Mechanical Design*, 147(4):041703, 2025.
- Hoogeboom, E., Peters, J., Van Den Berg, R., and Welling, M. Integer discrete flows and lossless compression. *Advances in Neural Information Processing Systems*, 32, 2019.
- Jo, J. and Hwang, S. J. Generative modeling on manifolds through mixture of riemannian diffusion processes. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 22348–22370, 2024.
- Jo, J. and Hwang, S. J. Continuous diffusion model for language modeling. *Advances in Neural Information Processing Systems*, 38:97394–97430, 2026.
- Karri, A., Stump, G., McComb, C., and Song, B. Hu-ver: The hyform uncrewed vehicle engineering repository. *Journal of Mechanical Design*, 147(4):044505, 2025.
- Kroo, I. A quasi-procedural, knowledge-based system for aircraft design. In *Aircraft Design, Systems and Operations Conference*, pp. 4428, 1994.
- Lopez-Paz, D. and Oquab, M. Revisiting classifier two-sample tests. In *International Conference on Learning Representations*, 2017.
- Lou, A., Meng, C., and Ermon, S. Discrete diffusion language modeling by estimating the ratios of the data distribution. *International Conference on Machine Learning*, 2024.
- Lueckmann, J.-M., Goncalves, P. J., Bassetto, G., Öcal, K., Nonnenmacher, M., and Macke, J. H. Flexible statistical inference for mechanistic models of neural dynamics. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/addfa9b7e234254d26e9c7f2af1005cb-Paper.pdf.
- Lueckmann, J.-M., Boelts, J., Greenberg, D., Goncalves, P., and Macke, J. Benchmarking simulation-based inference. In Banerjee, A. and Fukumizu, K. (eds.), *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pp. 343–351. PMLR, 13–15 Apr 2021. URL <https://proceedings.mlr.press/v130/lueckmann21a.html>.
- Lukaczyk, T. W., Wendorff, A. D., Colonna, M., Economon, T. D., Alonso, J. J., Orra, T. H., and Ilario, C. Suave: an open-source environment for multi-fidelity conceptual vehicle design. In *16th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, pp. 3087, 2015.
- Martins, J. R. and Kennedy, G. J. Enabling large-scale multidisciplinary design optimization through adjoint sensitivity analysis. *Structural and Multidisciplinary Optimization*, 64(5):2959–2974, 2021.
- Papamakarios, G. and Murray, I. Fast ε -free inference of simulation models with bayesian conditional density estimation. *Advances in neural information processing systems*, 29, 2016.
- Papamakarios, G., Sterratt, D., and Murray, I. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd international conference on artificial intelligence and statistics*, pp. 837–848. PMLR, 2019a.
- Papamakarios, G., Sterratt, D., and Murray, I. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 837–848. PMLR, 2019b.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4195–4205, 2023.
- Raymer, D. *Aircraft design: a conceptual approach*. American Institute of Aeronautics and Astronautics, Inc., 2012.
- Regenwetter, L., Abu Obaideh, Y., Chiotti, F., Lykourantzou, I., and Ahmed, F. Bikebench: A bicycle design benchmark for generative models with objectives and constraints. In *NeurIPS 2025 Datasets and Benchmarks Track*, 2025.
- Sahoo, S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.
- Shen, Y. and Alonso, J. J. Performance evaluation of a graph neural network-augmented multi-fidelity workflow for predicting aerodynamic coefficients on delta wings at low speed. In *AIAA SciTech 2025 Forum*, pp. 2360, 2025.
- Shi, J., Han, K., Wang, Z., Doucet, A., and Titsias, M. Simplified and generalized masked diffusion for discrete data. *Advances in neural information processing systems*, 37:103131–103167, 2024.

- Smart, J. T. and Alonso, J. J. Configuration selection via self-supervised, performance-weighted generative neural networks. In *AIAA SCITECH 2023 Forum*, pp. 0669, 2023.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Sung, N., Spreizer, S., Elrefaie, M., Samuel, K., Jones, M. C., and Ahmed, F. Blendednet: A blended wing body aircraft dataset and surrogate model for aerodynamic predictions. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89237, pp. V03BT03A049. American Society of Mechanical Engineers, 2025.
- Tejero-Cantero, Á., Durkan, C., Lueckmann, J.-M., Deistler, M., Gonçalves, P. J., Greenberg, D. S., Macke, J. H., et al. sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, 5(52):2505, 2020. doi: 10.21105/joss.02505. URL <https://doi.org/10.21105/joss.02505>.
- Thomas, O., Dutta, R., Corander, J., Kaski, S., and Gutmann, M. U. Likelihood-free inference by ratio estimation. *Bayesian Analysis*, 17(1):1–31, 2022.
- Tran, D., Vafa, K., Agrawal, K., Dinh, L., and Poole, B. Discrete flows: Invertible generative models of discrete data. *Advances in Neural Information Processing Systems*, 32, 2019.
- Vardhan, H., Kumari, N., Volgyesi, P., and Sztipanovits, J. Fusion of ml with numerical simulation for optimized propeller design. In *2024 IEEE Workshop on Design Automation for CPS and IoT (DESTION)*, pp. 22–27. IEEE, 2024.
- von Kaenel, R., Rizzi, A., Oppelstrup, J., Goetzendorf-Grabowski, T., Ghoreyshi, M., Cavagna, L., Bérard, A., et al. Ceasiom: simulating stability & control with cfd/csm in aircraft conceptual design. In *26th international congress of the aeronautical sciences, ICAS*, volume 1, pp. 14–19, 2008.
- Weilbach, C. D., Harvey, W., and Wood, F. Graphically structured diffusion models. In *International Conference on Machine Learning*, pp. 36887–36909. PMLR, 2023.
- Zhu, C., Yu, R., Feng, S., Burchfiel, B., Shah, P., and Gupta, A. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.

A. Algorithms

Algorithm 1 Train: Mixed Diffusion Transformer (MixeDiT)

Color-coded: **Topology (Discrete)**, **Observations (Continuous)**

Require: Initial point $\mathbf{u}$, model $s_\phi(\tau_{t_\tau}, \mathbf{x}_{t_x}, t_\tau, t_x)$, vocabulary size $\mathcal{T} + 1$, discrete time distribution $q(t_\tau)$, pre-computed $\{\alpha_{i/K}, \rho_{i/K}\}_{i=0}^K$, learning rate λ , loss weight γ . (Refer to B.1 for additional details.)

- 1: **for** each epoch **do**
- 2: Sample batch, B , of designs $\{\Theta_b\}_{b=1}^B = \{\theta_b, \tau_b\}_{b=1}^B$ and corresponding observations $\{\mathbf{x}_b\}_{b=1}^B$
- 3: Define end points of the bridge process: $\tau_b^0 \leftarrow \mathbf{u}_b$ and τ_b^1
- 4: Sample $t_\tau \sim q(t_\tau)$, $t_x \sim \mathcal{U}(0, 1)$, and $\mathbf{z}_b^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Simulation-Free Interpolation (See Jo & Hwang (2026)):
 Compute $(\alpha_{t_\tau}, \rho_{t_\tau}) \leftarrow \text{INTERPOLATE}(\{\alpha_{i/K}, \rho_{i/K}\}_{i=0}^K)$; $\Phi_b^0 \leftarrow (\cos^{-1}(\tau_b^0, \tau_b^1))_{b=1}^B$
 $\mu_b^{t_\tau} \leftarrow \left(\frac{\alpha_{t_\tau}}{\sin \Phi_b^0} \tau_b^1 + \sqrt{1 - \alpha_{t_\tau}^2} - \frac{\alpha_{t_\tau} \cos \Phi_b^0}{\sin \Phi_b^0} \tau_b^0 \right)_{b=1}^B$
 Discrete (Topology) Interpolant:
 $\tau_b^{t_\tau} \sim \mathcal{N}_{\mathbb{S}^{d-1}}(\mu_b^{t_\tau}, \rho_{t_\tau}^2 \mathbf{I}_{\mathcal{T}+1})$
- 6: Continuous (Obs.) Interpolant:
 $\mathbf{x}_b^{t_x} \leftarrow \alpha(t_x) \mathbf{x}_b^1 + \beta(t_x) \mathbf{z}_b^0$
- 7: Model Evaluation:
 $(\mathbf{p}_{\tau_t}^\phi, \epsilon_{\mathbf{x}_t}^\phi) = s_\phi(\tau_b^{t_\tau}, \mathbf{x}_b^{t_x}, t_\tau, t_x)$
- 8: $\ell_b^{RD}(\phi) = -\frac{1}{q(t_\tau)} \log \langle \mathbf{p}_{\tau_t}^\phi, \tau_b^1 \rangle$, and $\ell_b^{CSM}(\phi) = \|\epsilon_{\mathbf{x}_t}^\phi - \epsilon\|_2^2$
- 9: $\mathcal{L}^{MixeDiT}(\phi) = \frac{1}{B} \sum \gamma \ell_b^{RD} + \frac{1}{B} \sum (1 - \gamma) \ell_b^{CSM}$
- 10: $\phi_{i+1} \leftarrow \phi_i - \lambda \nabla_\phi \mathcal{L}^{MixeDiT}(\phi)$
- 11: **end for**
- 12: **Return:** ϕ

Algorithm 2 Sample: Mixed Diffusion Transformer (MixeDiT)

Color-coded: **Topology (Discrete)**, **Observations (Continuous)**

Require: Initial points $\mathbf{u}$ and $\mathbf{z}^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, trained model $s_\phi(\tau_{t_\tau}, \mathbf{x}_{t_x}, t_\tau, t_x)$, vocabulary size $\mathcal{T} + 1$, Number of sampling steps M , noise scheduler σ_t . (Refer to B.1 for additional details.)

- 1: $t_\tau = 0$, $t_x = 0$
- 2: $\tau^{t_\tau} \leftarrow \mathbf{u}$, $\mathbf{x}^{t_x} = \mathbf{z}^0$
- 3: **for** $m=1$ to M **do**
- 4: $\mathbf{w}_\tau \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{\mathcal{T}+1})$, and $\mathbf{w}_x \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: $(\mathbf{p}_{\tau_t}^\phi, \epsilon_{\mathbf{x}_t}^\phi) = s_\phi(\tau^{t_\tau}, \mathbf{x}^{t_x}, t_\tau, t_x)$
- 6: Evaluate Drift Terms:
 $\eta_\phi \leftarrow \text{Equation (8)}$
 $\mathbf{d}_{t_x} \leftarrow \text{Equation (9)}$
- 7: Update Equations:
 (Note: Fix $\mathbf{x}^{t_x} = \mathbf{x}^{\text{condition}}$ and $t_x = 1$ to condition on observations)
 $\tau^{t_\tau} \leftarrow \exp_{\tau^{t_\tau}} \left(\eta_\phi \delta t + \sigma_t \sqrt{\delta t} \mathbf{w}_\tau \right)$
 $\mathbf{x}^{t_x} \leftarrow \mathbf{x}^{t_x} + \mathbf{d}_{t_x} \delta t + \sigma_t \sqrt{\delta t} \mathbf{w}_x$
 $t_\tau = t_\tau + \delta t$
 $t_x = t_x + \delta t$
- 8: **end for**
- 9: $p_\tau = \pi^{-1}(\tau^{t_\tau})$ (Inverse diffeomorphism from sphere to simplex)
- 10: $\tau = \text{argmax}(p_\tau)$
- 11: **Return:** $\tau, \mathbf{x}^{t_x}$

Algorithm 3 MaskeDiT Training

Require: Dataset $\{\theta^{(i)}, \mathbf{x}^{(i)}, \tau^{(i)}\}_{i=1}^N$, noising process (SDE) defined in terms of $\alpha(t)$ and $\beta(t)$, minimum time is $T_{\min}$, batch size B , MaskeDiT simformer f_ω . See App. C.

- 1: **for** each epoch **do**
- 2: Sample batch $[\theta^{(b)}, \mathbf{x}^{(b)}, \tau^{(b)}]_{b=1}^B$
- 3: Build joint space $\mathbf{z} \leftarrow [\theta; \mathbf{x}]$
- 4: Sample diffusion timestep batch $t_b \sim \mathcal{U}(T_{\min}, 1)$
- 5: Sample condition mask M_C from the mixture

$$M_C \sim \frac{1}{5} \text{Bernoulli}(0.3) + \frac{1}{5} \text{Bernoulli}(0.7) + \frac{1}{5} \delta_{\mathbf{1}_{d_\theta} + d_x} + \frac{1}{5} \delta_{[\mathbf{0}_{d_\theta}, \mathbf{1}_{d_x}]} + \frac{1}{5} \delta_{[\mathbf{1}_{d_\theta}, \mathbf{0}_{d_x}]},$$

- 6: Build topology mask: For each component i in τ , $M_\tau(i) = 0$ if component i is in $\tau^{(b)}$.
 - 7: Build combined mask $M = (1 - M_C)M_\tau$
 - 8: Sample noise $\epsilon \sim \mathcal{N}(0, I)$ and apply forward diffusion $\mathbf{z}_t = \alpha(t)\mathbf{z}_1 + \beta(t)\epsilon$
 Predict the noise using the Simformer $\epsilon^\omega(\mathbf{z}, M_C, M, t)$
 - 9: **for** each $\mathbf{z}, M_C, M$ in batch **do**
 - 10: $t_{\text{embedding}} \leftarrow \text{GaussianEmbedding}(t)$
 - 11: $\text{ID}_{\text{embedding}} \leftarrow \text{Repeat}(\text{ID})$
 - 12: Construct conditional embedding:
 - 13: **for** each variable j **do**
 - 14: **if** $M_C(j) = 0$ **then**
 - 15: $\mathbf{c}_j \leftarrow \text{Enc}_w(\mathbf{z}_j)$
 - 16: **else**
 - 17: $\mathbf{c}_j \leftarrow 0$
 - 18: **end if**
 - 19: **end for**
 - 20: $v_{in} \leftarrow [t_{\text{embedding}}; \text{ID}_{\text{embedding}}; \mathbf{z}; \mathbf{c}]$
 - 21: $\epsilon \leftarrow \text{Transformer}(v_{in}, \text{Attention Mask} = M \otimes M)$
 - 22: **end for**
 - 23: Compute MaskeDiT loss $\mathcal{L}^{\text{MaskeDiT}} = \frac{M \|\epsilon^\omega(\mathbf{z}, M_C, M, t) - \epsilon\|_2^2}{\sum_i M_i}$
 - 24: Update parameters ω using gradient descent
 - 25: **end for**
-

Algorithm 4 MaskeDiT sampling process

Require: Trained MaskeDiT simformer f_ω , diffusion process defined as in App. C, steps K , condition mask $M_C = \mathbf{1}(\text{condition})$ and corresponding $\mathbf{z}$ values (note joint space $\mathbf{z} = [\theta; \mathbf{x}]$), topology τ

- 1: Build topology mask: For each component i in τ , $M_\tau(i) = 0$ if component i is in τ .
- 2: Sample initial noise $\mathbf{z}_K \sim \mathcal{N}(0, I)$
- 3: Impose conditioning $\mathbf{z}_K \leftarrow M_C \odot \mathbf{z}_K + (1 - M_C) \odot \mathbf{z}_K$
- 4: Set step size $\Delta t \leftarrow -1/K$
- 5: **for** $k = 0$ to $K - 1$ **do**
- 6: Compute time

$$t_k \leftarrow 1 - \frac{k}{K}(1 - T_{\min})$$

- 7: Predict noise $\hat{\epsilon}_k = \epsilon^\omega(\mathbf{z}, M_C, M, t)$
- 8: Convert to score

$$s^\omega \leftarrow -\frac{\hat{\epsilon}_k}{\beta(t_k)}$$

- 9: Sample Gaussian noise $e_k \sim \mathcal{N}(0, I)$
 - 10: Euler–Maruyama update $\mathbf{z}_{k-1} \leftarrow \mathbf{z}_k + \sigma(t_k)^2 s^\omega \Delta t + \sigma(t_k) \sqrt{-\Delta t} e_k$
 - 11: Impose conditioning $\mathbf{z} \leftarrow M_C \odot \mathbf{z} + (1 - M_C) \odot \mathbf{z}_{k-1}$
 - 12: **end for**
 - 13: Return x_0
-

B. MixeDiT: Additional Details

B.1. RDLM Head

Training (Alg. 1): We define $q(t_\tau)$ to up-weight training the middle part of the diffusion process. We use the unnormalized CDF $Q(t_\tau)$:

$$Q(t_\tau) = \begin{cases} w_{\text{low}} \cdot t_\tau & \text{if } t_\tau \leq 0.3 \\ w_{\text{low}} \cdot 0.3 + w_{\text{high}} \cdot (t_\tau - 0.3) & \text{if } 0.3 < t_\tau \leq 0.75 \\ w_{\text{low}} \cdot a + w_{\text{high}} \cdot (0.75 - 0.3) + w_{\text{low}} \cdot (t_\tau - 0.75) & \text{if } t_\tau > 0.75 \end{cases}$$

where $w_{\text{high}} = 1.0$ and $w_{\text{low}} = 10^{-4}$. To sample from $q(t_\tau)$, we first evaluate $Q(1) = \int_0^1 q(s)ds$, then sample $u \sim \mathcal{U}(0, Q(1))$ and solve for $Q(t_\tau) = u$.

The INTERPOLATE function of Alg. 1 relies on pre-computing parameters of a Riemannian normal before training. To compute these values, please refer to Alg. 3 of Jo & Hwang (2026). This interpolation enables us to sample directly from $\tau_b^{t_\tau} \sim p(\tau_b^{t_\tau} | \tau_b^0, \tau_b^1)$ rather than needing to simulate the full bridge process at every training step to get the interpolant (label).

Sampling (Alg. 2): The drift (Equation 11 from Jo & Hwang (2026)), is adapted for our single sequence length use-case:

$$\eta_\phi = \sum_{k=1}^d \langle \mathbf{p}_{\tau_t}^\phi, \mathbf{e}_k \rangle \gamma_t \frac{\cos^{-1} \langle \tau^{t_\tau}, \mathbf{e}_k \rangle (\mathbf{e}_k - \langle \tau^{t_\tau}, \mathbf{e}_k \rangle \tau^{t_\tau})}{\sqrt{1 - \langle \tau^{t_\tau}, \mathbf{e}_k \rangle^2}}, \quad (8)$$

where $\mathbf{e}_k$ is the one-hot vector with the k^{th} element set to 1, and $\gamma_t = \sigma_t^2 / \int_t^T \sigma_s^2 ds$.

Scheduler. We employ the geometric scheduler (Jo & Hwang, 2026) to define the noise evolution over the time horizon $t \in [0, 1]$. The noise rate $\beta(t)$ is interpolated geometrically between the initial rate $\beta_0 = 0.001$ and final rate $\beta_f = 0.2$.

Additional hyperparameters: We set the noise scheduler $\sigma_t = 2.5$ and the loss weighting parameter $\gamma = 0.9$ to put more emphasis on the discrete head. Gamma was selected through a grid search, evaluating topology cross-entropy over the validation set, where this choice of gamma emerged as the best trade-off in the sweep.

B.2. Continuous Diffusion Head

To evaluate the drift at time t_x and corresponding $\mathbf{x}^{t_x}$, we evaluate $\epsilon_{\mathbf{x}_t}^\phi$ from the model and then:

$$\mathbf{d}_{t_x} = \frac{\dot{\alpha}(t_x)}{\alpha(t_x)} \mathbf{x}^{t_x} - \frac{\epsilon_{\mathbf{x}_t}^\phi}{\beta(t_x)} \left(\beta(t_x)^2 \frac{\dot{\alpha}(t_x)}{\alpha(t_x)} - \beta(t_x) \dot{\beta}(t_x) + \frac{1}{2} \sigma_t \right). \quad (9)$$

We set $\alpha(t_x) = t_x$ and $\beta(t_x) = \sqrt{1 - t_x^2}$ (Ho et al., 2020).

B.3. MixeDiT Architecture

We build from the same architecture as in Jo & Hwang (2026)². Therefore, the model is based on a diffusion transformer architecture (Peebles & Xie, 2023) with rotary positional embeddings (Su et al., 2024). However, unlike the original work of Jo & Hwang (2026), we introduce an additional continuous input and a corresponding continuous timestep. We embed the continuous input into a hidden dimension of 768 using a linear layer and sum it with the discrete embedding to form the transformer input. For the time conditioning, the discrete and continuous timesteps are embedded independently, concatenated, and projected via an MLP to form a shared conditioning vector. This vector modulates the DiT blocks via Adaptive LayerNorm (AdaLN), while the input sequence is processed with rotary positional embeddings. Finally, a dual-head linear output layer splits the latent representation back into corresponding discrete logits and continuous outputs.

Model Hyperparameters: Vocab size: 144 (+1 mask token); Batch size: 256; Learning rate: 3e-4; EMA decay: 0.9999; Gradient clipping: 1.0; Hidden size: 768; Heads: 12; DiT blocks: 12.

²Their code base is here: <https://github.com/harryjo97/RDLM>.

C. MaskeDiT

Simformer (Gloeckler et al., 2024) models the joint distribution $p(\theta, \mathbf{x})$ using a score-based diffusion model in which a transformer network estimates the time-dependent score (Weilbach et al., 2023). Each data point is represented as a sequence of tokens corresponding to parameters and simulator outputs. Each token embedding is formed by concatenating a learned identity embedding ($d_{\text{id}} = 64$), the feature value embedding ($d_{\text{value}} = 64$), and a learned condition embedding ($d_{\text{cond}} = 32$) that indicates whether the variable is observed. Sinusoidal encodings are applied to the identity and condition components, resulting in a total model dimension of 160 per token.

The transformer consists of 16 layers with 2-head self-attention (key size 10), residual connections, and feed-forward MLP blocks with widening factor 3 and GELU activations. A Gaussian Fourier time embedding of dimension 64 conditions the network on the diffusion time variable and is injected as contextual information at each layer. The final score estimate is produced by a linear projection applied token-wise.

Conditional inference is enabled through binary condition masks, which are sampled uniformly at random during training, allowing the model to learn scores for the joint, posterior, likelihood, and other conditional distributions within a single architecture - see Alg. 3 for details.

The topology mask is applied to both rows and columns of the attention matrix, ensuring that absent components neither attend to nor receive attention from present ones. Components with $M_{\tau,i} = 0$ remain in the token sequence with zeroed embeddings. This preserves batching and positional structure, though excluding such tokens could reduce compute and is a potential direction for future refinement.

The diffusion process is defined with $t=0$ as Gaussian noise, and $t=1$ as the clean data. We define a Gaussian probability path, using $\mathbf{z} = [\theta, \mathbf{x}]$,

$$\mathbf{z}_t = \alpha(t)\mathbf{z}_0 + \beta(t)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (10)$$

with $\alpha(0) = 0, \beta(0) = 1$ and $\alpha(1) = 1, \beta(1) = 0$. α is the mean coefficient and β is the noise standard deviation, with $\alpha(t) = t$ and $\beta(t) = \sqrt{1 - t^2}$. This is a variance preserving SDE corresponding to a DDPM-style forward corruption process in continuous time (Ho et al., 2020) - though note that α , the mean coefficient, and β , the noise, here are not the same as the per-step α_k and β_k notation used in DDPM papers. Training minimizes a noise-prediction denoising score matching loss, which is equivalent to learning the score

$$s^\omega(x_t, t) = -\epsilon^\omega(\mathbf{z}_t, t)/\beta(t). \quad (11)$$

At inference, samples are generated by initializing from the Gaussian prior ($t = 0$) and integrating the reverse-time SDE using Euler-Maruyama

$$d\mathbf{z}_t = \mathbf{d}_{t\mathbf{z}} dt + \sigma(t)dW_t, \quad (12)$$

where the drift is calculated as in Eq. 9 and the second term modulates the contribution of the score. The diffusion coefficient $\sigma(t)$ adds stochasticity to the sampling. Unlike in DDPM (Ho et al., 2020), we explicitly control σ to trade off between sample diversity and fidelity to the mean of the target distribution.

We use $\sigma_{\min} = 10^{-4}$, $\sigma_{\max} = 15.0$, and sampling occurs with 500 diffusion steps. The model is trained for 500 epochs with batch size 256 using the Adam optimizer at a learning rate of 10^{-4} . Model at the epoch with the lowest validation loss is selected, using a train/test split of 20221 training points to 10057 test points. An exponential moving average (EMA) of the model parameters is maintained during training to improve stability and sampling quality, with an EMA decay weighting of 0.999.

D. Further Information on Case Studies

D.1. Case Study A: Higher mass for a fixed topology, fixed state of charge and fixed drag

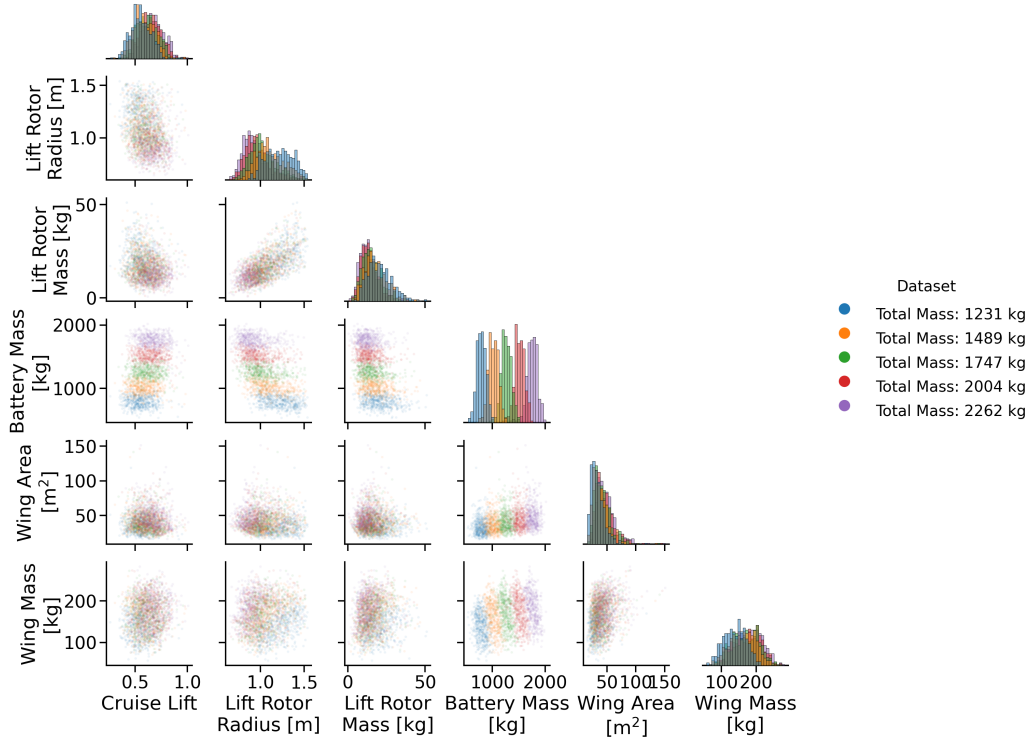


Figure 13. Case Study A: The pairplot for several selected variables for increasing mass.

Hypothesis. Increasing the mass requires the production of *more lift*, while constraining state of charge requires the aircraft to remain efficient. This could manifest in a *greater wingspan*, which would lead to decreased induced drag and an *increase in wing area*. We would also expect the *battery mass to increase* in order to power the larger aircraft.

We see a few clear trends from Fig. 13. First is the increase in battery mass with aircraft mass in order to achieve the final state of charge required. We also see the corresponding increase in the average cruise lift, though the distributions are broad due to a wide variety of designs possible. We also see the expected increase in wing mass. Wing area increases as a greater lift is required to balance the greater mass. We also see a decrease in lifting rotor radius as the mass increases. Though one might expect the lifting rotor radius to increase in order to maintain disk loading (the force carried per lifting rotor disk area), because the required thrust is higher, the RPM of the lifting rotor increases. This higher RPM leads to greater tip Mach numbers for larger rotors, so this tip Mach number limit is the constraint that forces smaller rotors for heavier aircraft. A corresponding decrease in lifting rotor mass is also seen.

When comparing the 3D renders of individual samples (Fig. 14), we see that the primary driver of aircraft mass is battery mass, with the geometry barely changing as mass is increased. However, we notice an increasing wingspan, as expected, as well as the aforementioned shrinking of lifting rotor blade radius.

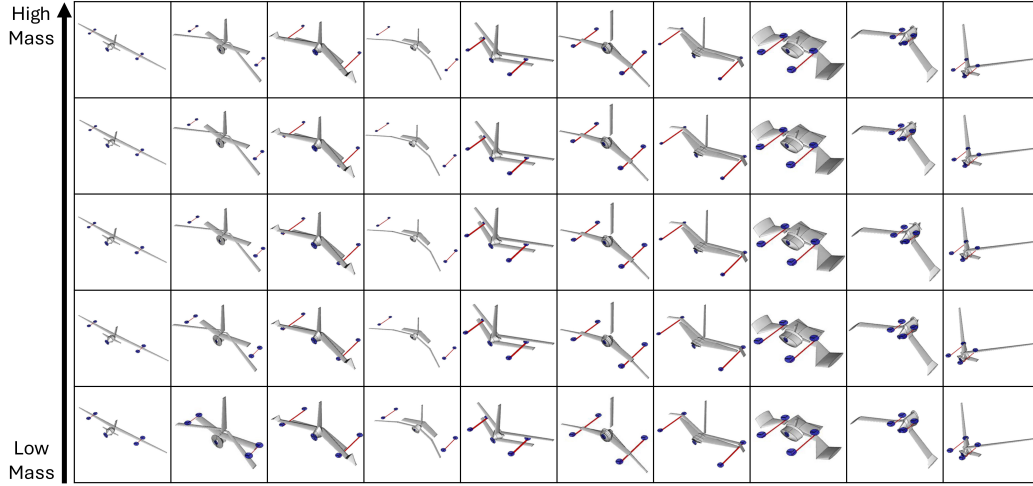


Figure 14. Case Study A: Note the correlation between seeds. This collage highlights the diversity of available design choices, whereby there are significant differences in battery mass across these designs despite similar CAD representations.

D.2. Case Study B: Increasing the lift of monoplanes versus biplanes for a fixed wing mass

Hypothesis. *Drag should increase with increasing lift with both topologies, though the biplanes should on average have a greater total wingspan.*

In this case we take advantage of the masking diffusion model to directly compare different topologies. In Fig. 15, we notice several trends. Overall, we see clear clustering both by mass and by topology. As expected, lower lift leads to lower mass designs, regardless of topology, with the distribution being bimodal between the lowest lift designs and the higher lift designs. The same trends are seen in battery mass. Interestingly, the near-quadratic relationship between lift coefficient and drag coefficient is recovered - a well known physical law saying that drag is approximately proportional to the square of lift ($C_D = C_{D,0} + KC_L^2$). This also leads to drag following the lift trend, and the lifting rotor exhibiting a bimodal distribution following the lift-based trends discussed in Case A in D.1. However, clustering occurs on topological lines when considering total wingspan, with the total wingspan of the biplane configurations being greater than the total wingspan of the monoplane configurations.

When considering the 3D samples in Fig. 9, we see that though the wingspan increases in order to increase lift coefficient while maintaining wing mass, we see that the wings become skinnier and the span increases.

D.3. Case Study C: Lower mass with other x fixed

Hypothesis. We would require *greater battery mass* and *greater wing mass* to lift a heavier aircraft.

In Fig. 16, we see the same increase in battery mass with mass that we saw before, correlated with an increase in the wing mass, which was also predicted. In the lifting rotor radius, we see the same trends with shortening to maintain tip Mach number with increased mass. This is further amplified by the ability to increase the number of lifting rotors, which allows the figure of merit to be kept constant while shortening rotor radius. The cross correlations between wing mass and lifting rotor mass, and battery mass and lifting rotor mass both have two clear clusters, one for monoplanes and one for biplanes. Note how effectively the multinomial distribution of the number of blades is captured. We see a greater number of forward rotors as the mass increases, as larger rotors lead to greater drag and greater mass than using a greater number of small rotors. The lifting rotors also increase correspondingly with mass to maintain the climb efficiency (a greater number of lifting rotors increases the figure of merit). The heavier designs also benefit from the extra lift and drag of a second wing (the lift and drag coefficients are per unit wing area), though a single wing design is still the norm overall.

We clearly see the increasing numbers of second wings in the renderings of aircraft samples in Fig. 17, as well as larger wings leading to higher wing masses.

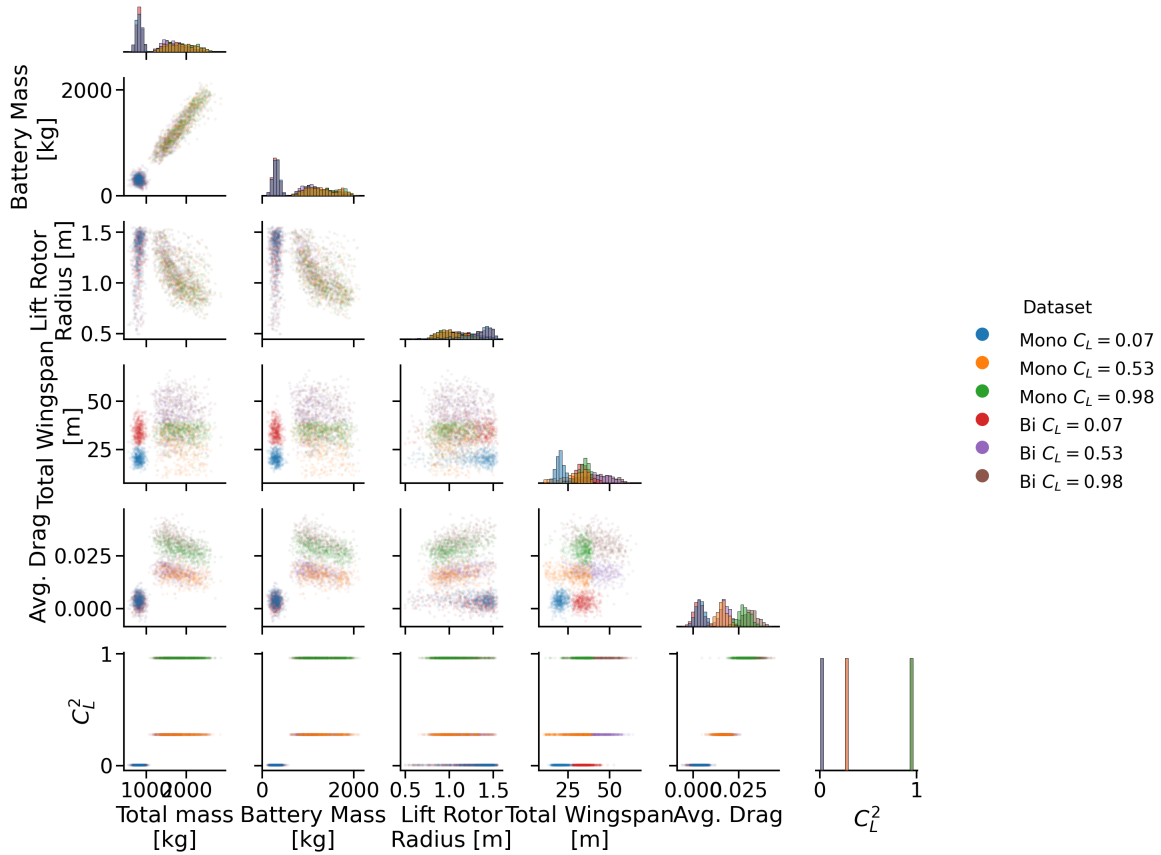


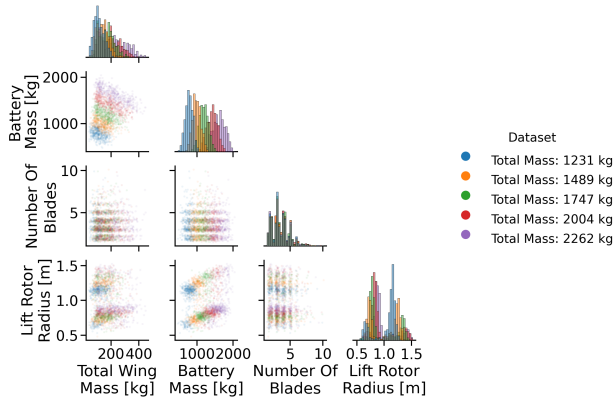
Figure 15. Case Study B: The pairplot for several selected variables comparing monoplanes and biplanes with increasing lift.

D.4. Case Study D: Lower drag with other x fixed

Hypothesis. For greater drag with the same lift, we would expect *smaller wingspans* to increase induced drag. Designs with greater drag should have *more components*.

For a greater drag, we see a increase in the number of forward rotors, as this increases rotor-wing interaction as well as profile drag caused by the rotors (Fig. 18). We also see that draggier designs typically have a second wing, due to the greater profile drag, and, depending on the wing arrangement, greater induced drag. We see the decreased wingspans predicted, alongside increased wing mass at the wing chord and thickness increases, with distributions being clustered into monoplanes and biplanes. This is the same with the lifting rotor mass, as lifting rotors act as dead mass in cruise. Likewise the lifting rotor radius increases with increasing drag.

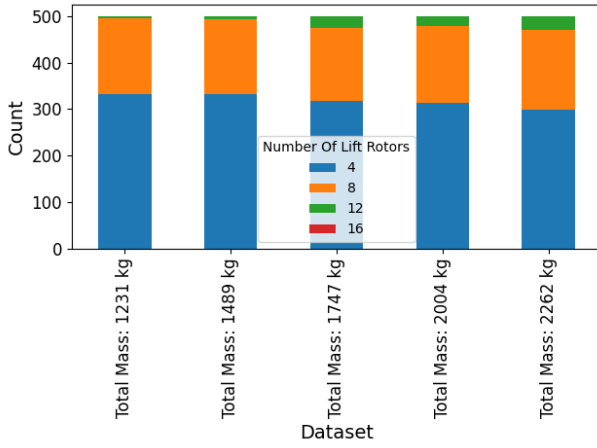
The renderings of aircraft samples in Fig. 12 show several important trends with increasing drag coefficient. The first is the decrease in wing efficiency, with wings going from long and skinny (to maximize aspect ratio, and hence minimize induced drag) to thicker and stubbier. The number of lifting rotors (which are dead mass during cruise) and number of wings notably increases too, increasing drag due to wake and nonbeneficial interactions between wings. This is particularly prevalent in the number of wings stacked vertically very close together.



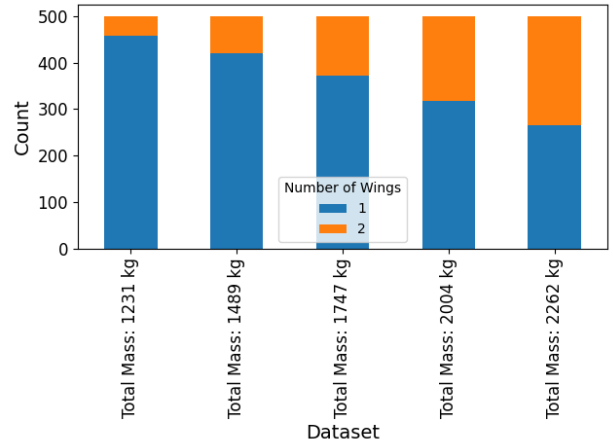
(a) Pairplot of continuous variables.



(c) Number of forward rotors with increasing mass.



(b) Number of lifting rotors with increasing mass.



(d) Number of wings with increasing mass.

Figure 16. Case Study C: Several selected variables for increasing mass without fixing topology.

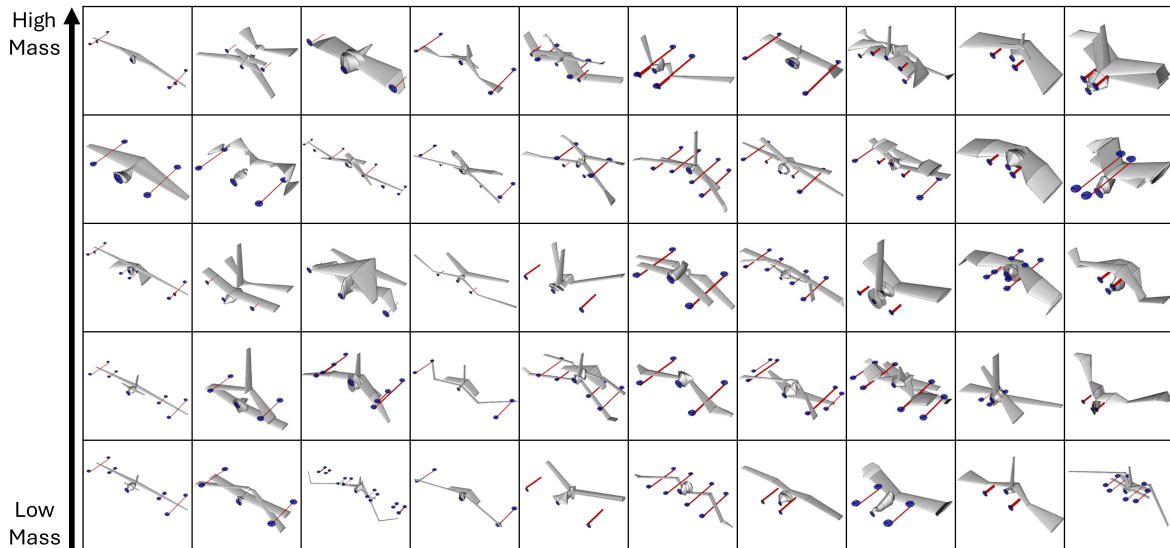
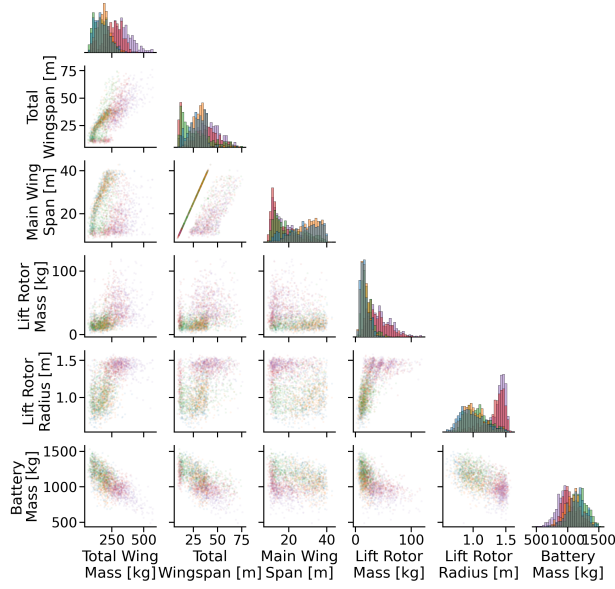
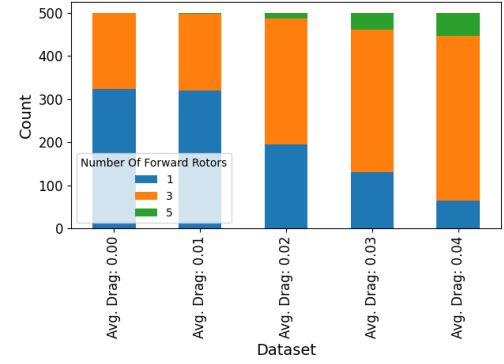


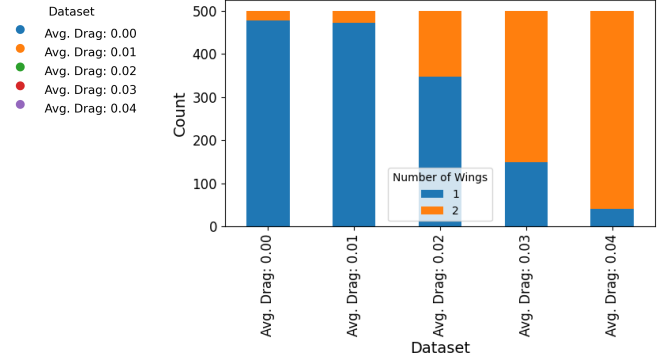
Figure 17. Case Study C: renderings for increasing mass and allowing variable topology.



(a) Pairplot of continuous variables.



(b) Number of forward rotors with increasing drag.



(c) Number of wings with increasing drag.

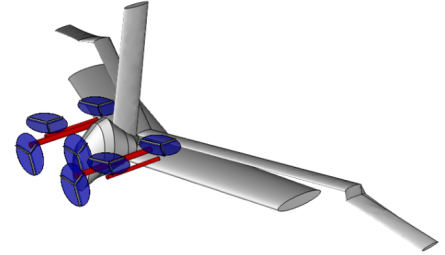
Figure 18. Case Study D: Several selected variables for increasing drag without fixing topology.

E. Dataset Generation: Additional details

Aircraft Design Tree



CAD Visualization



SUAVE Outputs

Observation	
Cruise Average Drag	0.0892
Cruise Average Lift	0.6706
Cruise Angle of Attack	3.117 (deg.)
Fuselage Mass	63.63 (Kg)
Lift Rotors Mass	18.43 (Kg)
Total Wing Mass	137.27 (Kg)
Total Mass	1083.44 (Kg)
Battery State of Charge	9.05 (%)

Figure 19. Full design tree for an example aircraft.

Tree structure We generate a dataset of 30,276 eVTOL aircraft, described by the parameters θ and performance characteristics x (see symbolic prior, E.1). The interface to the simulator is a design tree, which is the full combinatorial design space of eVTOL configurations. The root represents the full vehicle architecture, which branches into propulsion, structural, aerodynamic, energy storage, and avionics subsystems. Each subsystem is further refined into concrete components, such as airframe members and wings, subcomponents such as propeller rotors, until reaching atomic parameters such as geometric dimensions and masses. This tree forms a typed, hierarchical JSON schema that functions like a generative grammar, with the full prior described in Tab. 2. The table defines a top-level design class (*Aircraft*) together with a set of component classes. This schema forms a structural design tree in which every node corresponds to a continuous, discrete, enumerated, or structural choice. Each field in the schema is annotated with metadata describing allowable ranges, maximum or minimum numbers of list elements, or enumerated sets, ensuring that the generative process generates valid aircraft while avoiding premature designer bias. Most prominently, the method fixes the maximum quantities of components and subcomponents. The framework then assigns a probability distribution to every choice point, using priors selected using domain knowledge to define a probabilistic generative model over the entire design tree. Integer parameters are chosen from Poisson distributions, with the lambda parameter based on minimum and maximum values, whereas continuous choices are drawn from appropriate uniform priors, determined from domain knowledge. Discrete choices are drawn from uniform categorical priors, and Boolean choices are sampled from a uniform Bernoulli distribution.

Simulator There exist several aircraft simulation tools used by aircraft designers in order to size aircraft during the conceptual phase (Lukaczyk et al., 2015; Gratz et al.; von Kaenel et al., 2008). Each of these tools, originally developed for the purposes of multidisciplinary optimization, allow an aircraft design to be evaluated and analyzed. We use SUAVE, an environment which excels in designing unconventional aircraft, suitable for dataset generation where breadth is required. Given an aircraft design and mission (flight path), SUAVE performs three key coupled analyses sequentially: stability, aerodynamic and propulsion. The aircraft longitudinal static stability is found and the dynamic stability is determined using a three-equation approximation. The aerodynamics are decomposed into lift and drag calculations. The lift is found using a Gaussian Process based surrogate model of an aerodynamic simulation of the aircraft, in this case a Vortex Lattice Method (VLM), though interfaces to higher fidelity computational fluid dynamics (CFD) platforms are available. The drag

predictions from the VLM are augmented with semi-empirical viscous drag estimates. When modeling propeller-driven aircraft or rotorcraft, the propellers and lifting rotors are automatically designed based on optimizing for a set of user-defined design requirements. The propellers are simulated using Blade Element Momentum Theory (BEMT) with a wake model, and batteries are modeled using a semi-empirical approach that considers discharge behavior, heat transfer, and cell aging, as described in [Clarke & Alonso \(2021\)](#). More details on SUAVE can be found in [Lukaczyk et al. \(2015\)](#).

Verification and visualization To prevent analyzing clearly infeasible aircraft without overrestricting the design space, and to avoid wasting compute time, we verify the structure of the aircraft before simulating the aircraft. Firstly, we built a simple parametric CAD modeling tool using the python package CADQuery ([Cowden & CadQuery Contributors](#)). We use that tool to perform a Boolean intersection check to ensure that propellers do not intersect with wings or fuselages. The CAD model can be rendered for visualization at this stage. Secondly, we reject any aircraft meeting the following criteria (based on our own domain knowledge): wingspans over 50 m, aspect ratios over 30, and any wing with a chord less than 0.3 m at any point. Finally, we implement a simple structural check to reject aircraft with infeasible wings. We model the wing as a cantilever beam of half-span $L = b/2$ subjected to a uniform distributed load $w = W/(4L)$, where W , the aircraft weight. The resulting root bending moment is approximated as

$$M_{\text{wing}} = \frac{wL^2}{2}. \quad (13)$$

The wing cross section is approximated as a solid square beam with side length $s = c_{\text{root}}(t/c)$, and the maximum bending stress is estimated as

$$\sigma_{\text{wing}} = \frac{M_{\text{wing}}(s/2)}{I}, \quad (14)$$

and compared against an allowable aluminum stress.

Lift-rotor support booms are modeled as solid circular cantilever beams of length L_b with a tip load equal to the motor weight, $W_b = m_b g$. This leads to a maximum bending stress of

$$\sigma_b = \frac{W_b L_b r}{I_b}, \quad I_b = \frac{\pi r^4}{4}, \quad (15)$$

where r is the boom radius. Designs for which the estimated bending stress in either the wing or boom exceeded the allowable stress are rejected.

SUAVE exhibits known failure modes on portions of the prior’s support, and we explicitly detect and reject such cases before including them in the dataset. The most common issues are:

- Structural infeasibility or component intersections, dealt with via the checks described previously.
- Non-convergent mission simulations, where the solver fails to identify a feasible trajectory, where we reject unconverged simulations.
- Vortex Lattice Method (VLM) mesh failures, which may occasionally produce nonphysical artifacts such as negative drag or unrealistically large drag values, where we reject designs that lead to nonphysical drag or list values.

Simulation uncertainty Including internal fuselage and wing cross-sections directly in the generative parameterization would expand the design space from 136 to approximately 331 continuous variables, substantially increasing model complexity while contributing limited value for the conceptual-level trends we study. In conceptual design, certain levels of design detail are not known but are required for simulation. This design uncertainty manifests in the the exact wing and fuselage shapes, where component-level properties such as length are known at the conceptual stage. We therefore treat these internal geometries as simulator-side uncertainty rather than primary inference targets, sampling them at run time from the same priors used during dataset generation. The leaves that describe the internal cross sections of the wings and fuselages are removed from the design trees after simulation. During sampling, the internal cross sections are re-drawn from the priors.

Post-processing The observations from the simulator are appended as their own component in the design tree. A global set of features is constructed, one for each leaf, by enumerating every possible leaf in the maximal design tree. Each feature’s key corresponds to a path of the form

root/component/subcomponent/property

The flattening of a particular JSON design tree into this feature space is equivalent to mapping its leaves onto the maximal tree. Designs with fewer components simply leave the corresponding entries empty with preference for the first instance of a component, so a design with only one wing fills `wing/1/*` while all `wing/2/*` fields remain empty.

Resampling When simulating aircraft generated via MaskeDiT/MixeDiT rather than via the design tree generator, there will be fields missing, as mentioned in **Simulation uncertainty** above. However, the design tree generator framework can also complete partial designs. The same generator and prior is used to sample only the missing parameters while holding the rest of the parameters fixed.

Limitations of dataset approach There are several noteworthy limitations of our dataset. We encompass a very large breadth of designs, many of which are unreasonable aircraft and would not be able to fly outside of the simulator. More traditional-looking aircraft can be found through more constrained dataset generation, or stronger conditioning at inference time. Moreover, it is unlikely that SUAVE is capable of accurately capturing such uncommon aircraft, particularly in terms of the full set of aerodynamic interactions occurring between components. SUAVE is principally build on low fidelity simulation tools and empirical correlations. For example, the lift prediction errors are typically on the order of 10%, but errors are much larger outside of normal operating conditions (Botero et al., 2022). The complex flowfields, including nonlinear phenomena like separated flow, are likely present for many of these aircraft. This means that errors likely significantly exceed 10% for lift prediction.

E.1. Symbolic Prior

The table below highlights the structure of the symbolic prior that we use to generate the dataset. The way to read the table is to start from the `Aircraft` class and visit each of its components. For example, one first samples a list of `MainWing` types. For each `MainWing` in the list, one then samples the corresponding list of parameters. For our dataset, we implemented rejection sampling for designs that do not meet our template of a maximum of two wings, each with a maximum of two lift prop arms, and one forward prop arm. Because the prior intentionally encodes only coarse domain knowledge—such as approximate parameter scales rather than a joint feasible set—many sampled designs fail structural or stability checks. The topology prior, induces substantial variation in pass rates across configurations. In the final dataset, topologies have a median of 182 feasible samples, with the ten most common topologies accounting for 22.4% of all valid designs.

Table 2. Description of probabilistic generator (prior) over aircraft model.

COMPONENT	DESCRIPTION	DISTRIBUTION
RotorTemplate tip.radius hub.radius number.of.blades design.tip.mach design.altitude design.thrust	Propeller data class.	$\mathcal{U}(0.5, 1.5)$ $\mathcal{U}(0.1, 0.2)$ $\mathcal{C}(p), p = 0.25$ classes: (2,5) $\mathcal{U}(0.5, 0.8)$ $\mathcal{U}(500.0, 2000.0)$ $\mathcal{U}(1500.0, 3000.0)$
ForwardRotor origin.z.rel	Inherits from <code>RotorTemplate</code> .	$\mathcal{U}(-0.5, 0.5)$
LiftRotor origin.z.rel	Inherits from <code>RotorTemplate</code> .	$\mathcal{U}(0.1, 0.5)$
PropArm origin.x.rel origin.y.rel origin.z.rel length.nose.tail width fineness	Propeller arm base class.	$\mathcal{U}(-0.5, 0.5)$ $\mathcal{U}(0.1, 1.0)$ $\mathcal{U}(-0.5, 0.5)$ $\mathcal{U}(0.05, 0.3)$ $\mathcal{U}(0.05, 0.3)$ $\mathcal{U}(0.5, 1.0)$
ForwardPropArm length.main	Inherits from <code>PropArm</code> .	$\mathcal{U}(0.5, 3.0)$
LiftRotorArmSymmetric	Inherits from <code>PropArm</code> .	

Continued on next page

“Do Diffusion Models Dream of Electric Planes?” Discrete and Continuous SBI for Aircraft Design

Table 2 – continued from previous page

COMPONENT	DESCRIPTION	DISTRIBUTION
length.main		$\mathcal{U}(1.0, 8.0)$
Wing origin.x.rel origin.z.rel	Wing base class. Origin is relative from the nose of the aircraft.	$\mathcal{U}(0.0, 1.0)$ $\mathcal{U}(-0.5, 0.5)$
WingCrossSection twist.deg root.chord.percent dihedral.outboard.deg sweep.quarter.chord.deg thickness.to.chord	Wing cross section base class.	$\mathcal{U}(-5.0, 5.0)$ $\mathcal{U}(0.1, 1.0)$ $\mathcal{U}(-5.0, 5.0)$ $\mathcal{U}(-45.0, 45.0)$ $\mathcal{U}(0.06, 0.24)$
RootMainWingCrossSection	Inherits from WingCrossSection.	
TipMainWingCrossSection	Inherits from WingCrossSection.	
InnerMainWingCrossSection percent.span.location	Inherits from WingCrossSection.	$\mathcal{U}(0.01, 0.99)$
MainWing span.proj chord.root root.cross.section inner.cross.sections tip.cross.section prop.arms	Inherits from Wing. Type: RootMainWingCrossSection List of InnerMainWingCrossSection Type: TipMainWingCrossSection List of PropArm	$\mathcal{U}(10.0, 40.0)$ $\mathcal{U}(0.5, 5.0)$ Length of list $\sim \mathcal{Po}(1.5)$ Length of list $\sim \mathcal{Po}(1.5)$
VerticalTail area.reference taper aspect.ratio sweep.quarter.chord.deg thickness.to.chord	Inherits from Wing.	$\mathcal{U}(0.5, 5.0)$ $\mathcal{U}(0.1, 1.0)$ $\mathcal{U}(1.0, 5.0)$ $\mathcal{U}(0.0, 30.0)$ $\mathcal{U}(0.06, 0.24)$
HorizontalTail area.reference taper aspect.ratio sweep.quarter.chord.deg thickness.to.chord dihedral.deg	Inherits from Wing.	$\mathcal{U}(0.0, 30.0)$ $\mathcal{U}(0.1, 1.0)$ $\mathcal{U}(3.0, 15.0)$ $\mathcal{U}(-30.0, 30.0)$ $\mathcal{U}(0.06, 0.24)$ $\mathcal{U}(-5.0, 5.0)$
FuselageCrossSection percent.x.location percent.z.location	Abstract base class. Percent of fuselage length	$\mathcal{U}(0.0, 1.0)$ $\mathcal{U}(-0.1, 0.1)$
NoseFuselageCrossSection height width	Inherits from FuselageCrossSection.	$\mathcal{U}(0.05, 0.2)$ $\mathcal{U}(0.05, 0.2)$
TailFuselageCrossSection height width	Inherits from FuselageCrossSection.	$\mathcal{U}(0.05, 0.2)$ $\mathcal{U}(0.05, 0.2)$
InnerFuselageCrossSection height width	Inherits from FuselageCrossSection.	$\mathcal{U}(0.1, 3.0)$ $\mathcal{U}(0.1, 3.0)$
InnerFuselagePowerSeriesCrossSection	Inherits from FuselageCrossSection.	
Fuselage length.cabin area.wetted nose.cross.section tail.cross.section prop	Abstract base class. Type: NoseFuselageCrossSection Type: TailFuselageCrossSection Type: ForwardRotor (Main prop)	$\mathcal{U}(1.5, 4.0)$ $\mathcal{U}(15.0, 30.0)$
RandomFuselage inner.cross.sections	Inherits from Fuselage. List of InnerFuselageCrossSection	Length of list $\sim \mathcal{Po}(4.5)$
PowerSeriesFuselage widest.point.x.rel widest.point.z.rel widest.point.height widest.point.width power.front.height power.front.width power.rear.height power.rear.width front.cross.sections rear.cross.sections	Inherits from Fuselage. List of InnerFuselagePowerSeriesCrossSection List of InnerFuselagePowerSeriesCrossSection	$\mathcal{U}(0.1, 0.9)$ $\mathcal{U}(-0.1, 0.1)$ $\mathcal{U}(0.1, 3.0)$ $\mathcal{U}(0.1, 3.0)$ $\mathcal{U}(0.0, 1.0)$ $\mathcal{U}(0.0, 1.0)$ $\mathcal{U}(0.0, 1.0)$ $\mathcal{U}(0.0, 1.0)$ Length of list $\sim \mathcal{Po}(2.5)$ Length of list $\sim \mathcal{Po}(2.5)$
Battery battery.type voltage mass	Data class. Choice: battery.types	“Li-Ion” $\mathcal{U}(300.0, 800.0)$ $\mathcal{U}(300.0, 2000.0)$
Aircraft main.wings horizontal.tails vertical.tails fuselage	Data class. List of MainWing List of HorizontalTail List of VerticalTail Type: Fuselage	Length of list $\sim \mathcal{Po}(1.5)$ truncated to be $\in [1, 2]$ Length of list $\sim \mathcal{Po}(0.5)$ truncated to be $\in [0, 1]$ Length of list $\sim \mathcal{Po}(0.5)$ truncated to be $\in [0, 1]$ $\mathcal{U}\{\text{RandomFuselage, PowerSeriesFuselage}\}$

Continued on next page

Table 2 – continued from previous page

COMPONENT	DESCRIPTION	DISTRIBUTION
battery	Type: Battery	
forward.prop	Type: RotorTemplate	
lift.prop	Type: RotorTemplate	

F. Posterior Predictive Checking and Full Pipeline

After building a dataset of the format in App. E, and training the MixeDiT and MaskeDiT models, we can sample using the process in Alg. 5. After sampling, we can optionally reconvert the designs into the tree format of the training dataset, as described in App. E. To perform posterior predictive checks, we can then run the designs through the simulator.

Algorithm 5 Sampling Procedure for Mixed Continuous–Discrete Aircraft Designs

Require: Performance requirements $\mathbf{x}_{\text{req}}$

- 1: Sample topology τ using MIXEDiT, which parametrizes $p(\tau \mid \mathbf{x}_{\text{req}})$.
- 2: Sample continuous parameters θ using MASKEDiT, which models $p(\theta \mid \tau, \mathbf{x}_{\text{req}})$.

Optional Post-Processing for PPC:

- 3: Clamp any out-of-distribution values to the nearest valid value (e.g., negative thickness $\rightarrow$ zero).
- 4: Reconstruct the full design tree representation.

Simulation:

- 5: For each sampled design, resample components not modeled by the simulator (wing sections, fuselage sections).
- 6: Run the resulting designs through the SUAVE simulator to obtain true performance values $\mathbf{x}_{\text{true}}$.

As can be seen from the full posterior in Fig. 20, the posterior distribution’s mean and the conditioning value are very similar, being almost indistinguishable in all plots. When looking at $\mathbf{x} = \mu + \sigma$ (Fig. 21) and $\mathbf{x} = \mu - \sigma$ (Fig. 22), we find that agreement is overall good. The largest difference in both off-mean cases is total wing weight, indicating that wing weight may be the variable that is least well predicted by the model. Note that when compared to the data, the posteriors clearly fit a subset of the training data for these cases, though in the case of $\mathbf{x} = \mu - \sigma$, this subset is small. The Posterior predictive checks confirm that the model reliably reproduces SUAVE-evaluated performance when conditioned on target specifications. In the $\mathbf{x} = \mu$ case, across the ten variables, the average L2 error is 5.3% and the square root of the average variance is 0.82, with variance dominated by cruise angle of attack and fuselage mass, and MSE dominated by lift-rotor mass. These checks demonstrate that the learned posterior is robust to the mismatch between the broad generative prior and the feasible design space.

G. Timing experiments

SUAVE is restricted to CPU-only hardware, and currently cannot be run with MPI. To benchmark SUAVE, we run 100 eVTOL simulations. In order to remove the cost of generating feasible designs from the SUAVE runtime, we run a single eVTOL case. Over the 100 runs, each simulation takes an average of 25.23 seconds, or a total time of 2523 seconds. The simulations were run on an Intel Xeon CPU E5-2680 v4 @2.40GHz with 14 cores and 28 logical processors, with 64 GB of RAM. The MixeDiT-MaskeDiT sampling was run on an NVIDIA A100-PCIE-40GB GPU, which for 100 generated designs took a time of 195.6 seconds, with the majority of time devoted to loading the model checkpoints.

H. Full comparison with NLE

In order to use MMD to compare across architectures, we perform a p-value test across the 10 topologies used, as recommended in the original MMD paper (Gretton et al., 2012). This is particularly important due to the unstable MMD value in the NLE case. Using a 5% rejection rate we find that the better MMD found in NLE is significant. From the table, the greatest disadvantage of the NLE approach is that it underperforms for some topologies, with a larger variation between the best case (minimum) marginal C2ST and worst case (maximum) marginal C2ST. Taking the standard deviations into account, the mean, median, maximum and minimum marginal C2ST scores are comparable between the two models. Therefore, we can say that MaskeDiT achieves comparable performance to NLE trained on each topology, while learning

Table 3. Likelihood comparison between generated samples $p_\phi(\mathbf{x}|\theta)$ and test data $p_{\mathcal{D}}(\mathbf{x}|\theta)$. Average is taken over 10 topologies with more than 200 samples in the test dataset, Mean and standard deviation $\mu \pm \sigma$ shown.

METRIC	MASKEDiT	NLE
MMD VALUE	0.0040 ± 0.0006	0.0021 ± 0.0007
JOINT C2ST	0.7773 ± 0.0395	0.7445 ± 0.0227
MEAN MARGINAL C2ST	0.6142 ± 0.0155	0.5961 ± 0.0105
MEDIAN MARGINAL C2ST	0.5973 ± 0.0299	0.5614 ± 0.0158
MAX MARGINAL C2ST	0.7699 ± 0.0762	0.8185 ± 0.0426
MIN MARGINAL C2ST	0.5065 ± 0.0247	0.4963 ± 0.0210

the significantly more complex joint distribution across all topologies. The NLE is implemented using the `sbi` package (Tejero-Cantero et al., 2020), using the package’s default hyperparameters. Three highly correlated variables are omitted from the C2ST calculation. Ten NLE models with 50k parameters each were trained. Scaling to 144 topologies would require 7.2M parameters, versus 5.1M parameters for the trained MaskediT model in this paper.

I. Related Work Cont.

Discrete SBI Simulation based inference (SBI) with discretized data is most commonly applied when the observations arise from a discretization of continuous features, such as temporal discretization, but recent work has broadened the range of applicable data types. Automatic Posterior Transformation (APT) extends earlier SNPE approaches by directly learning a mapping from data to the true posterior which can operate on raw time series data without requiring hand crafted summary statistics (Greenberg et al., 2019). Similarly, Gloeckler et al. (2024) demonstrate in their SBI framework that transformer architectures can successfully infer parameters from discretized and unevenly sampled time series, as illustrated through a Lotka–Volterra population model. Most similarly to our work, Boelts et al. (2022) introduce mixed neural likelihood estimators (MNLEs) for models with mixed discrete–continuous outputs, achieving improved likelihood estimation for neuroscience models like the drift–diffusion model by performing conditional density estimation. However, this approach assumes continuous inputs, so it not truly mixed SBI as we define it. It is worth noting that although not explicitly used for SBI, several flow based generative methods have been adapted specifically for discrete data, such as the work of Hoogeboom et al. (2019) and Tran et al. (2019).

Design generation in aircraft design SUAVE is a forward simulator and cannot be used for design generation or sampling; it evaluates performance only once a complete design is specified. Classical approaches to conceptual design generation fall into two main categories.

- (1) Optimization-based methods, which are powerful but typically applied at later design stages due to the size and complexity of the conceptual design space and because they return point solutions rather than distributions (Martins & Kennedy, 2021).
- (2) Knowledge-based engineering (KBE), which uses procedural, rule-based systems to emulate an engineer’s step-by-step reasoning. These methods were explored extensively in the 1990s but are widely regarded as inflexible and have seen limited development in recent decades (Kroo, 1994).

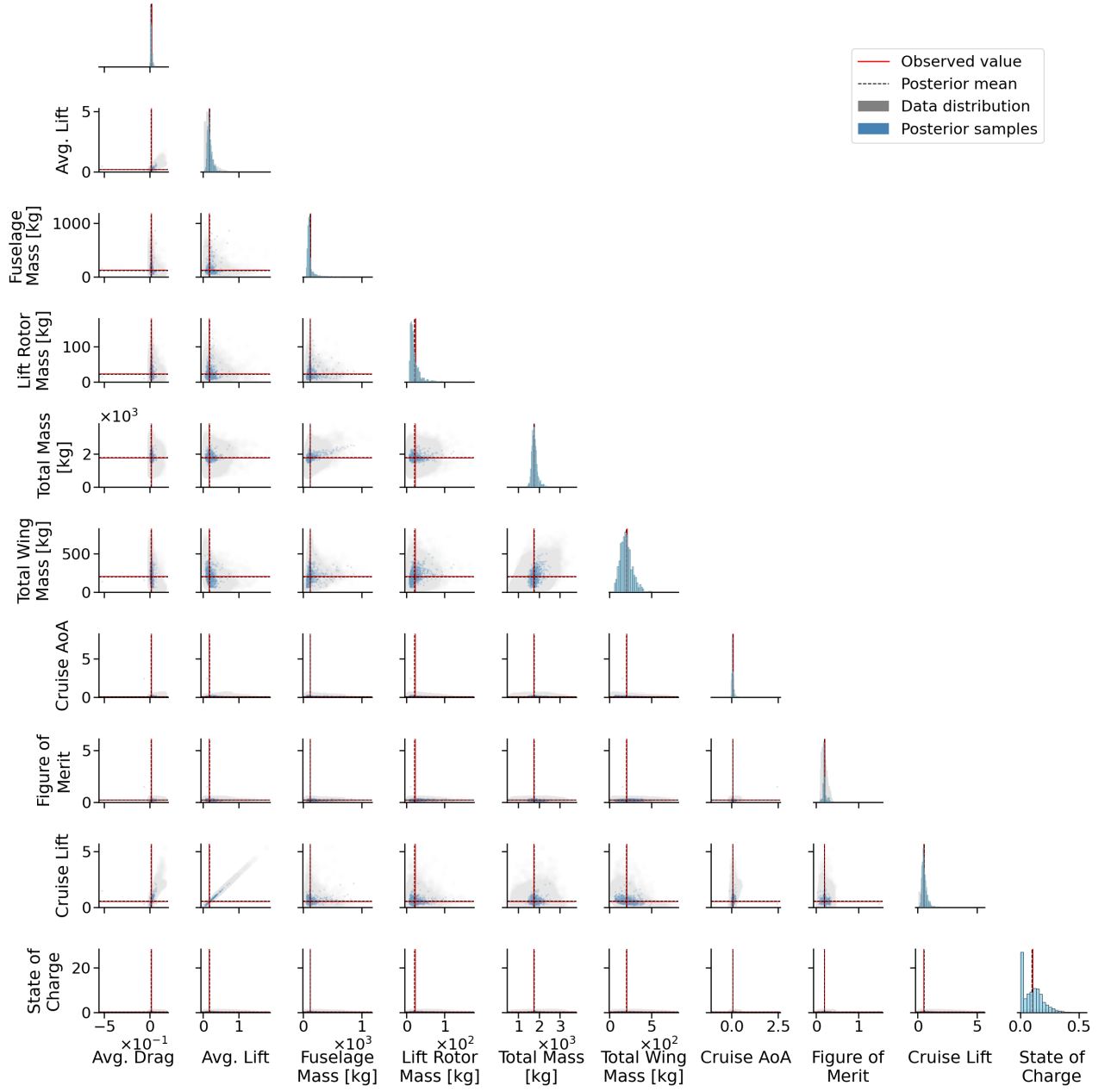


Figure 20. The full posterior predictive distribution over all ten $x = \mu$ variables. The average aircraft in the dataset, shown with the solid red line, is what the sampling from the architecture is conditioned on. When the samples are run through the simulator again, the distributions of x values obtained are shown in blue, with the dotted black line showing the means of those distributions. Data distributions are in gray.

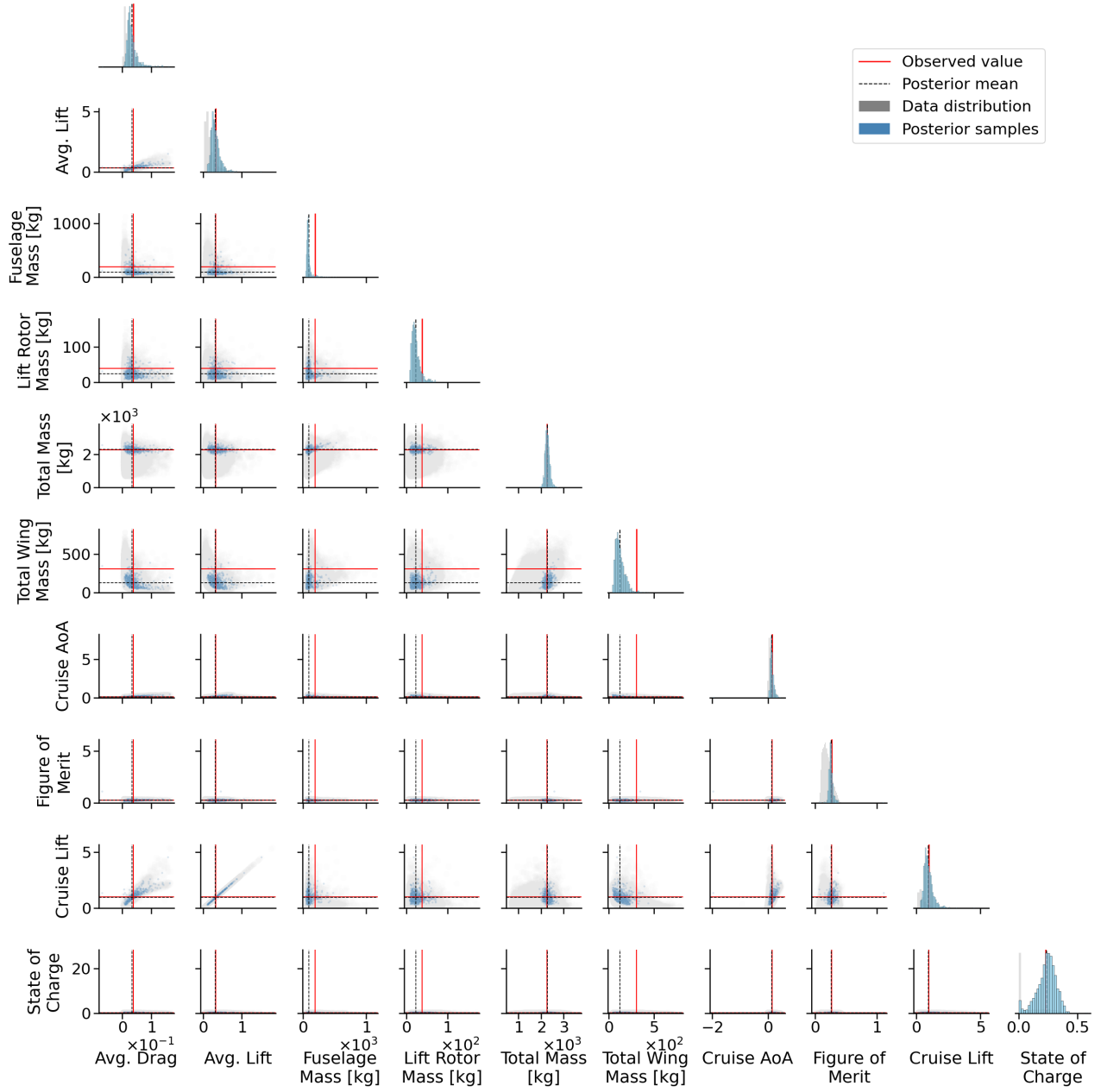


Figure 21. The full posterior predictive distribution over all ten $\mathbf{x} = \boldsymbol{\mu} + \boldsymbol{\sigma}$ variables.

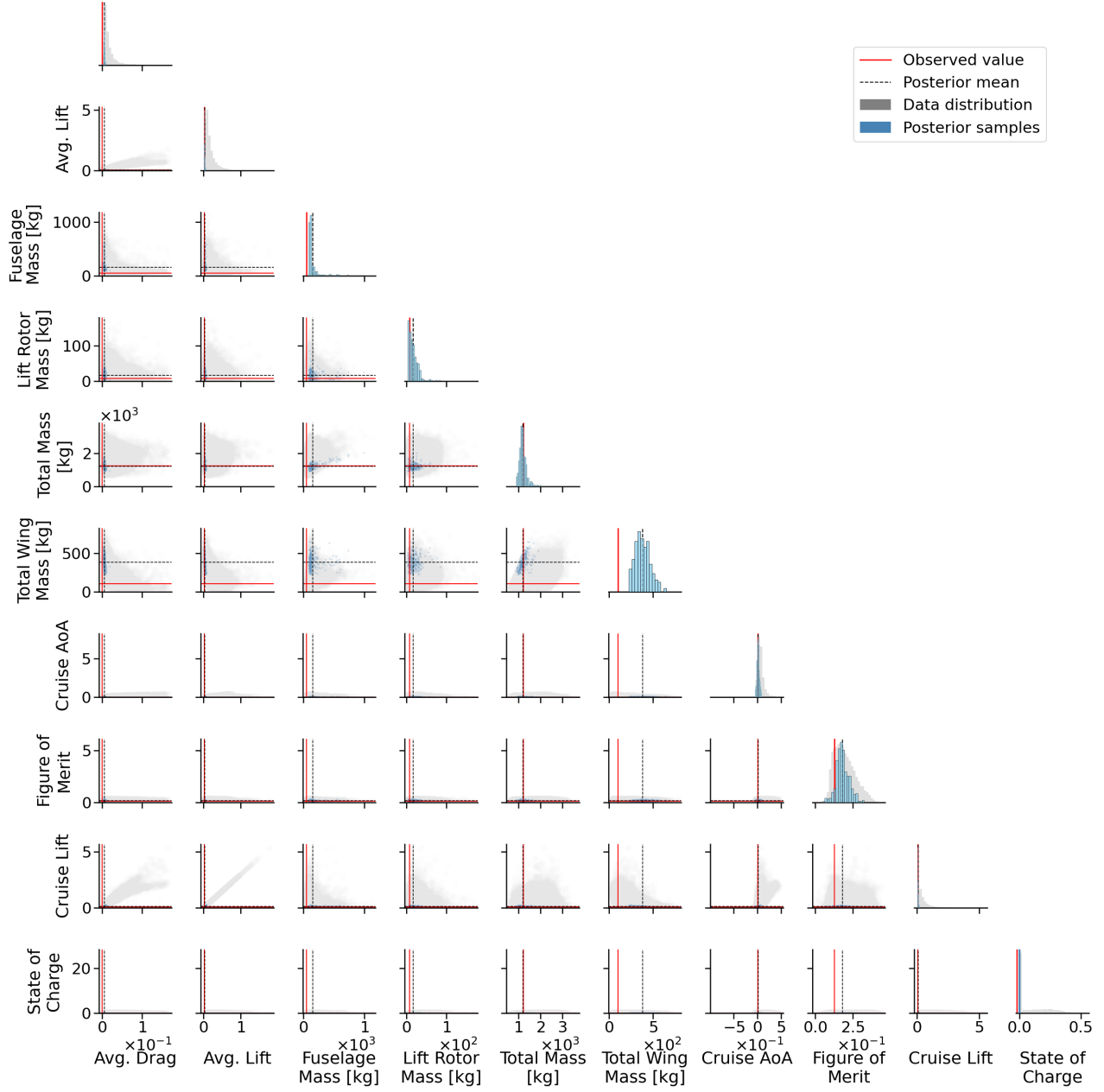


Figure 22. The full posterior predictive distribution over all ten $\mathbf{x} = \boldsymbol{\mu} - \boldsymbol{\sigma}$ variables.